

Vom Perzeptron zum Deep Network

Vorlesungsskript · Woche 1

Roter Faden der Veranstaltung. Die Vorlesung folgt dem ML-Lifecycle: *Modelle bauen* → *Modelle produktiv machen* → *Modelle verantworten* → Capstone. Dieses Skript legt das Fundament für alles Weitere: Jedes Modell, das später deployt, überwacht und erklärt wird, wird mit genau den hier behandelten Bausteinen trainiert.

Inhaltsverzeichnis

Vorbemerkung und Organisatorisches	5
1 Einordnung: Was ist Deep Learning – und warum jetzt?	6
1.1 Begriffsklärung: KI, Machine Learning, Deep Learning	6
1.2 Vom linearen Modell zur gelernten Merkmalsextraktion	6
1.3 Sichtbarer Fortschritt: Zehn Jahre in fünf Bildern	7
1.4 Warum jetzt? Die drei Treiber	8
1.5 Leitbeispiel: Handschrifterkennung (MNIST)	9
2 Vom Neuron zum Perzeptron	10
2.1 Das McCulloch-Pitts-Neuron	10
2.2 Das Perzeptron und seine Grenze: XOR	10
3 Das Multilayer-Perzeptron	11
3.1 Architektur und Vorwärtsrechnung	11
3.2 Aktivierungsfunktionen	12
3.3 Universelle Approximation und die Rolle der Tiefe	13
3.4 Ausgabeschichten und Loss-Funktionen	14
3.5 Das MLP in PyTorch	14
4 Gradientenbasiertes Lernen und Backpropagation	15
4.1 Lernen als Optimierungsproblem	15
4.2 Berechnungsgraphen und die Kettenregel	15
4.3 Vorwärts- oder Rückwärtsmodus?	16
4.4 Backpropagation für ein MLP	16
4.4.1 VJPs der Standardschichten	17
4.5 Allgemeine Graphen	18
4.6 Autograd in PyTorch	18
5 Optimierung: SGD, Momentum und adaptive Verfahren	19
5.1 Lernen ist nicht reine Optimierung	19
5.2 Stochastischer Gradientenabstieg	19
5.3 Was die Optimierung tiefer Netze schwer macht	19
5.4 Momentum und Nesterov-Momentum	20
5.5 Adaptive Lernraten: AdaGrad, RMSProp, Adam	21
5.6 Parameterinitialisierung	23
6 Regularisierung	23
6.1 Overfitting erkennen	24
6.2 Parameternormen: Weight Decay und L1	24
6.3 Early Stopping	24
6.4 Dropout	24
6.5 Rauschen und Datenaugmentation	25
6.6 Batch Normalization	25
6.7 Implizite Regularisierung durch SGD	26
6.8 Entscheidungsleitfaden: Welche Regularisierung wann?	26
7 Vanishing und Exploding Gradients	26
7.1 Analyse	27
7.2 Gegenmaßnahmen	27

8 Trainingsverläufe interpretieren**28**

Vorbemerkung und Organisatorisches

Organisatorisches – Woche 1

Termin	KW 41 (08.10), 4-SWS-Block (Theorie) und 2-SWS-Block (Lab)
Block	A – Vertiefung Deep Learning
Pflichtlektüre	[1]Goodfellow et al. (2016), Kap. 6 (<i>Deep Feedforward Networks</i>) und Kap. 8 (<i>Optimization for Training Deep Models</i>)
Vertiefung	[2]Kingma & Ba (2015): <i>Adam: A Method for Stochastic Optimization</i>
Alternativ	[3]Bishop & Bishop (2024), Kap. 6–9 · Murphy (2022), <i>PML: An Introduction</i> , Kap. 13 · [1]Goodfellow et al. (2016), Kap. 7 (Regularisierung)
Online	http://neuralnetworksanddeeplearning.com/chap1.html · https://playground.tensorflow.org
Lab	Einrichtung Python/Jupyter/PyTorch, MLP für MNIST, Architektur-Experimente

Notation

Symbol	Typ	Bedeutung
$x, \mathbf{x}, X$	Skalar, Vektor, Matrix	Fett = Vektor/Matrix; $\mathbf{x} \in \mathbb{R}^D$ ist ein Eingabevektor der Dimension D
$\boldsymbol{\theta} = \{\mathbf{W}_l, \mathbf{b}_l\}_{l=1}^L$	Parameter	Gewichtsmatrizen und Bias-Vektoren aller L Schichten
$\mathbf{a}_l = \mathbf{W}_l \mathbf{z}_{l-1} + \mathbf{b}_l$	Prä-Aktivierung	lineare Transformation in Schicht l
$\mathbf{z}_l = \varphi_l(\mathbf{a}_l)$	Aktivierung	Ausgabe der Schicht l ; $\mathbf{z}_0 = \mathbf{x}$
$\varphi(\cdot)$	Funktion	elementweise Aktivierungsfunktion (ReLU, Sigmoid, ...)
$\mathcal{L}(\boldsymbol{\theta})$	Skalar	Verlust-/Kostenfunktion auf dem Trainingsdatensatz $\mathcal{D} = \{(\mathbf{x}_n, \mathbf{y}_n)\}_{n=1}^N$
$\nabla_{\boldsymbol{\theta}} \mathcal{L}$	Vektor	Gradient des Verlusts nach den Parametern
$J_f(\mathbf{x}) = \partial f / \partial \mathbf{x}$	Matrix	Jacobi-Matrix von $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, Größe $m \times n$
ϵ (bzw. η)	Skalar	Lernrate (Schrittweite)
$\odot$	Operator	elementweises Produkt (Hadamard)

Die Notation folgt Murphy (2022) und ist mit Goodfellow et al. (2016) kompatibel; dort wird die Lernrate ϵ genannt, in PyTorch heißt sie `lr`.

1 Einordnung: Was ist Deep Learning – und warum jetzt?

1.1 Begriffsklärung: KI, Machine Learning, Deep Learning

Die drei Begriffe werden im Alltag oft synonym verwendet, bezeichnen aber ineinander geschachtelte Teilgebiete (Abb. 1):

Künstliche Intelligenz (KI)

Jede Technik, mit der ein Computer Verhalten zeigt, was beim Menschen als intelligent gilt. Einschließlich fest programmierter Regelsysteme, Suche und Planung. Ein Schachprogramm mit handgeschriebener Bewertungsfunktion ist KI, lernt aber nichts.

Machine Learning (ML)

Der Teil der KI, in dem das Verhalten nicht explizit programmiert, sondern aus Daten gelernt wird. Der Mensch legt fest, *welche* Merkmale ein Modell sieht (z.B. Zeilenabstand, Anzahl Links und Großbuchstaben-Anteil für einen Spam-Filter). Das Modell lernt anschließend nur noch, wie es diese Merkmale zu einer Entscheidung verrechnet.

Deep Learning (DL)

Der Teil des ML, in dem tiefe neuronale Netze Muster *direkt aus Rohdaten* extrahieren ohne vorherigen Merkmalsentwurf eines Menschen. Z.B. können Muster aus Pixeln, Audiosamples und Zeichenfolgen gewonnen werden.

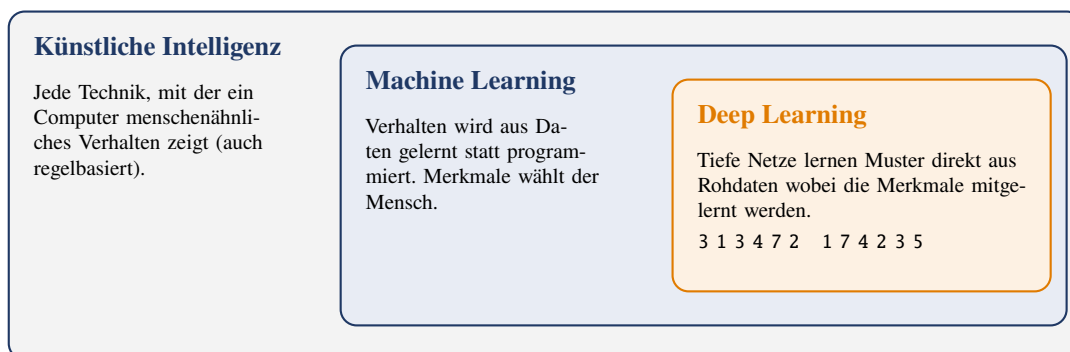


Abbildung 1: $KI \supset ML \supset DL$. Der entscheidende Schritt von ML zu DL ist nicht die Modellgröße, sondern der Wegfall des manuellen Merkmalsentwurfs: Das Netz sieht die Rohdaten (etwa Pixel handgeschriebener Ziffern) und lernt die Merkmale selbst.

Merke

Deep Learning heißt, einem Computer beizubringen, eine Aufgabe *direkt aus Rohdaten* zu lernen. Die zentrale Verschiebung gegenüber klassischem ML: Nicht nur der Klassifikator, sondern auch die Merkmale werden aus Daten gelernt (*Repräsentationslernen*).

1.2 Vom linearen Modell zur gelernten Merkmalsextraktion

Lineare Modelle wie etwa lineare Regression, logistische Regression, verallgemeinerte lineare Modelle sind gut verstanden, konvex und effizient zu fitten. Ihre zentrale Einschränkung ist die Annahme, dass die Abbildung von Eingabe x zu Ausgabe y linear in den Parametern ist. Der klassische Ausweg ist eine handgewählte Merkmalstransformation $\phi(x)$ (Basisfunktionen, Polynome, Kernel):

$$f(x; \theta) = W \phi(x) + b. \quad (1)$$

Das bleibt linear in $\theta = (W, b)$, verlagert aber die gesamte Modellierungsarbeit in die Wahl von ϕ . Genau dieser Merkmalsentwurf (*Feature Engineering*) ist in der Praxis das Problem: Er ist

- **zeitaufwendig:** für jede neue Domäne (Bilder, Audio, Sensordaten, Text) muss Expertenwissen in Merkmale übersetzt werden,

- **brüchig:** handgemachte Merkmale (etwa SIFT-Deskriptoren oder Kantenfilter für Bilder) funktionieren unter den Bedingungen, für die sie entworfen wurden, und brechen bei veränderter Beleuchtung, Perspektive oder Sensorik ein
- **nicht skalierbar:** die Merkmalsqualität wächst nicht mit der Datenmenge, sondern nur mit dem Aufwand, den ein Mensch investiert.

Deshalb die zentrale Frage welche mithilfe von Deep Learning versucht wird zu beantworten: *Können die zugrunde liegenden Merkmale direkt aus den Daten gelernt werden?* Der Kerngedanke ist, den Merkmals-extraktor selbst mit Parametern θ_2 auszustatten und mitzulernen:

$$f(\mathbf{x}; \theta) = \mathbf{W} \phi(\mathbf{x}; \theta_2) + \mathbf{b}. \quad (2)$$

Wendet man dieses Prinzip rekursiv an, entsteht eine Komposition aus L parametrisierten Funktionen:

$$f(\mathbf{x}; \theta) = f_L(f_{L-1}(\cdots f_1(\mathbf{x}) \cdots)), \quad (3)$$

wobei $f_l(\cdot) = f(\cdot; \theta_l)$ die Funktion der Schicht l ist. Ist die Komposition eine Kette wie in Gl. (3), spricht man von einem *Feedforward-Netz* bzw. *Multilayer-Perzeptron* (MLP). Allgemeiner lassen sich differenzierbare Bausteine zu beliebigen gerichteten azyklischen Graphen (DAGs) verknüpfen, welche die Grundlage für CNNs, Transformer und Graph-Netze bilden.

Merkmals-hierarchie. Viele Schichten in einem Netz dienen dabei einem bestimmten Zweck: Sie ermöglichen die Entstehung einer Hierarchie in welcher jede Schicht die Merkmale der vorherigen Schicht aufgreift und zu etwas komplexeren kombiniert. So ähnlich wie aus Buchstaben Wörter und aus Wörtern Sätze werden. Für ein auf Gesichtern trainiertes Netz lässt sich das z.B. sichtbar machen, indem man die Eingaben bestimmt, auf welche die einzelne Einheiten am stärksten reagieren (Abb. 2). Frühe Schichten reagieren auf Kanten und Linien, mittlere auf Augen, Nasen und Ohren, späte auf ganze Gesichtsstrukturen. Kein Mensch hat diese Hierarchie vorgegeben. Sie entsteht, weil sie die kompakteste Beschreibung der Trainingsdaten ist.

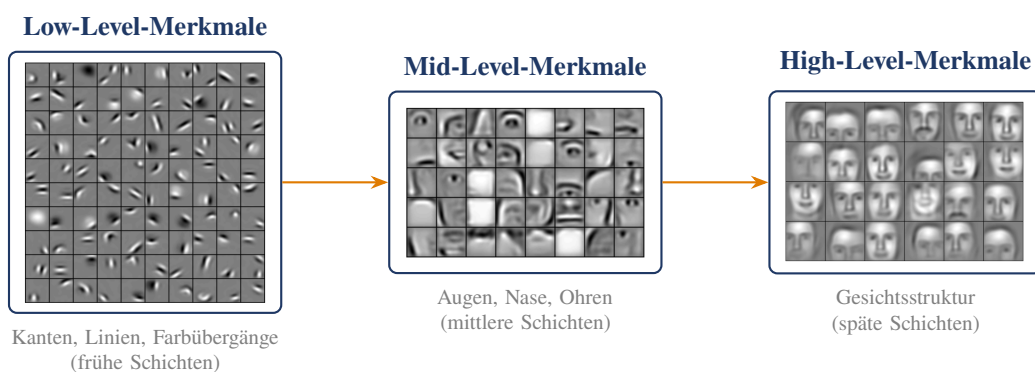


Abbildung 2: Gelernte Merkmals-hierarchie eines tiefen Netzes für Gesichtsbilder (nach [4]). Jede Schicht kombiniert die Merkmale der vorigen zu abstrakteren Mustern. Das Prinzip überträgt sich auf andere Domänen: Bei Audio entstehen Phoneme aus Frequenzmustern, bei Text Satzstrukturen aus Wortgruppen.

1.3 Sichtbarer Fortschritt: Zehn Jahre in fünf Bildern

Wie schnell sich das Feld bewegt, lässt sich zum Großteil an einer einzigen Aufgabe ablesen: Der Synthese von Gesichtern und Sprache:

Jahr	Meilenstein	Was möglich wurde
2015	Generative Adversarial Networks [5]	Verschwommene, aber erkennbare synthetische Gesichter mit rund 64×64 Pixeln
2018	StyleGAN (Karras, Laine, Aila)	Fotorealistische Gesichter in 1024×1024 Pixeln, von echten Fotos kaum unterscheidbar
2020	Synthetische Sprecher-Videos	Bewegte Personen mit synchronisierter, synthetischer Stimme („Deepfakes“)
2022	ChatGPT (GPT-3.5)	Dialogfähige Sprachmodelle für ein Massenpublikum
2023	GPT-4	Multimodale Modelle mit Leistung auf Niveau von Fachprüfungen
2025	Kompakte Modelle auf dem Endgerät	Modelle mit wenigen Mrd. Parametern laufen lokal auf dem Smartphone. Teilweise mit Benchmark-Werten über dem GPT-4 von 2023

Zwei Beobachtungen aus dieser Reihe sind für den Rest der Vorlesung wichtig.

Sprache hat die Interaktion verändert. Bis etwa 2020 waren Deep-Learning-Systeme Werkzeuge für Spezialisten: Man trainierte ein Modell für eine Aufgabe und band es in eine Anwendung ein. Mit den großen Sprachmodellen (LLMs) seit 2022 ist die natürliche Sprache selbst zur Schnittstelle geworden. Ein einziges Modell kann viele Aufgaben übernehmen und wird mittels einfacher Anweisungen gesteuert.

Skalierung läuft in zwei Richtungen. Die öffentliche Wahrnehmung ist auf immer größere Modelle in immer größeren Rechenzentren fokussiert. Parallel dazu hat sich die Effizienz kleiner Modelle so stark verbessert, dass Fähigkeiten, die 2023 ein Rechenzentrum erforderten, 2025 auf ein Mobiltelefon passen. Tabelle 1 zeigt ein Beispiel: ein offenes Modell mit 2,6 Mrd. Parametern gegen das ursprüngliche GPT-4 (geschätzt >1 Billion Parameter).

Tabelle 1: Benchmark-Vergleich LFM2-2.6B-Exp (Liquid AI, lokal auf dem Smartphone lauffähig) gegen GPT-4 (Original, 2023). Zahlen laut Herstellerangabe vom Dezember 2025; höhere Werte sind besser. Das kleine Modell liegt in 5 von 6 Benchmarks vorn. Bei breitem Expertenwissen (MMLU-Pro) jedoch deutlich hinten ==> Wissen skaliert mit Parametern, Anweisungsbefolgung nicht zwingend.

Benchmark	Misst	LFM2-2.6B	GPT-4
IFEval	Strikte Anweisungsbefolgung	88,13	79,0
Multi-IF	Komplexe Anweisungen	73,05	65,0
IFBench	Formatierungsgenauigkeit	44,40	24,0
GPQA	Fachliches Schließen (Promotionsniveau)	41,31	35,7
MMLU-Pro	Breites Expertenwissen	53,87	63,7
AIME25	Mathematik (Olympiadeniveau)	22,67	16,0

Für das KI-Engineering, das in der zweiten Hälfte der Vorlesung im Mittelpunkt steht, ist die zweite Beobachtung die relevantere: Modellgröße ist eine Entwurfsentscheidung mit Kosten (Latenz, Energie, Infrastruktur, Datenschutz), keine Qualitätsgarantie.

1.4 Warum jetzt? Die drei Treiber

Die Ideen hinter tiefen Netzen sind Jahrzehnte alt (Abb. 3): Das McCulloch-Pitts-Neuron zur Verarbeitung binärer Signale wurde 1943 vorgeschlagen; das stochastische Gradientenverfahren, mit dem bis heute praktisch jedes Netz trainiert wird, geht auf Robbins & Monro (1951) zurück; das Perzeptron mit lernbaren Gewichten folgte 1958, Backpropagation für mehrschichtige Netze in der „Mainstream“-Rezeption durch Rumelhart, Hinton & Williams 1986, und tiefe Faltungsnetze zur Ziffernerkennung (LeCun et al.) Ende der 1980er- und in den 1990er-Jahren. Die breite Nutzung setzte dennoch erst um 2010 ein. Zunächst in der Spracherkennung, dann 2012 in der Bilderkennung mit dem ImageNet-Ergebnis von Krizhevsky, Sutskever

& Hinton (*AlexNet*), das die Top-5-Fehlerrate von 26,2 % (bester klassischer Ansatz) auf 15,3 % senkte. Zuvor lag der Fortschritt bei etwa 2 Prozentpunkten pro Jahr. Warum diese Verzögerung von rund 25 Jahren zwischen Backpropagation und Durchbruch? Drei Faktoren mussten zusammenkommen:

1. **Big Data.** Große gelabelte Datensätze und allgegenwärtige digitale Datenerfassung: ImageNet umfasste bei Veröffentlichung (2009) rund 3,2 Mio. annotierte Bilder, Wikipedia und soziale Medien liefern Text in bislang ungekanntem Umfang. Erst damit lassen sich Modelle mit Millionen bis Milliarden Parametern trainieren, ohne dass sie die Trainingsdaten nur auswendig lernen.
2. **Hardware.** Graphics Processing Units (GPUs), ursprünglich für die Bildsynthese in Videospielen entwickelt, beschleunigen genau die massiv parallelen Matrix-Vektor-Operationen, aus denen Netze im Kern bestehen. AlexNet wurde 2012 auf zwei Consumer-GPUs trainiert; heutige Beschleuniger (GPUs, TPUs) sind um Größenordnungen schneller.
3. **Software.** Offene Bibliotheken wie TensorFlow, PyTorch und Keras liefern automatische Differenzierung, GPU-Kernel und fertige Bausteine. Ein Netz, das früher mühsam in C++ implementiert werden musste, entsteht nun in wenigen Zeilen. Dazu kamen methodische Verbesserungen: Die ReLU-Aktivierung entschärfte das Problem verschwindender Gradienten (Abschn. 7), Dropout und Batch Normalization machten tiefe Netze trainierbar (Abschn. 6), Adam machte das Training robust gegenüber der Wahl der Lernrate (Abschn. 5).

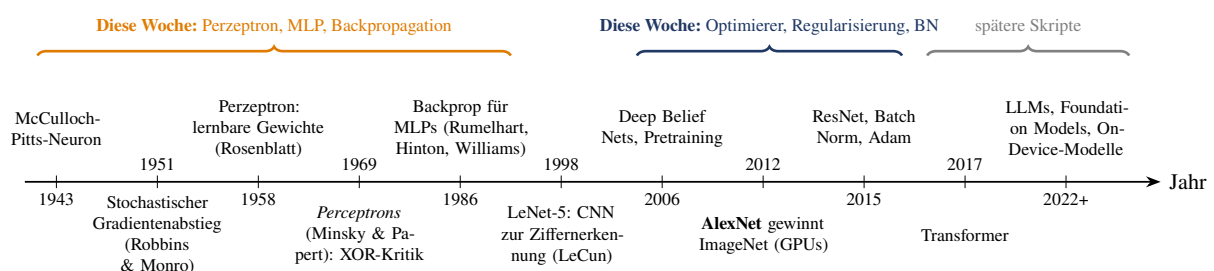


Abbildung 3: Zeitleiste ausgewählter Meilensteine. Woche 1 behandelt die Grundlagen (links) und die Trainingsmethoden der „modernen“ Phase (Mitte). Architekturen ab 2012 folgen in den Skripten *CNNs & Computer Vision* und *Sequenzmodelle & Transformer*.

Merke

Keiner der drei Treiber ist eine neue Idee über neuronale Netze. Der Durchbruch entstand, als Daten, Rechenleistung und Werkzeuge gleichzeitig ausreichten, um alte Ideen in großem Maßstab auszuprobieren. Das erklärt auch, warum diese Vorlesung so viel Gewicht auf das *Engineering* legt: Die Mathematik der Netze ist seit 1986 im Kern unverändert; was sich verändert hat, ist, wie man sie zuverlässig trainiert, ausrollt und überwacht.

1.5 Leitbeispiel: Handschrifterkennung (MNIST)

Als durchgängiges Beispiel dieses Skripts und des Labs dient der MNIST-Datensatz: 70 000 Graustufenbilder handgeschriebener Ziffern (60 000 Training, 10 000 Test), jeweils 28×28 Pixel, mit Labeln $y \in \{0, \dots, 9\}$ (Abb. 4). Für ein MLP wird jedes Bild zu einem Vektor $x \in \mathbb{R}^{784}$ abgeflacht; die Aufgabe ist eine 10-Klassen-Klassifikation. Damit ist MNIST zugleich das kleinste Beispiel für den Kern von Deep Learning aus Abb. 1: Das Netz erhält 784 Pixelwerte und sonst nichts. Alle Merkmale, die eine „3“ von einer „8“ unterscheiden, muss es selbst finden. (Eine feste Permutation aller Pixel würde die Leistung eines MLP nicht verändern, denn das Netz nutzt keine räumliche Nachbarschaft; genau das ändert sich mit CNNs im nächsten Skript.)

MNIST ist bewusst klein: Ein MLP erreicht nach zwei Epochen bereits rund 97 % Testgenauigkeit, sodass Sie im Lab in Minuten experimentieren können. Genau deshalb ist es das ideale Terrain, um die *Mechanik* des Trainings zu verstehen, bevor im Skript *CNNs & Computer Vision* die Architektur auf Bilder

zugeschnitten wird.

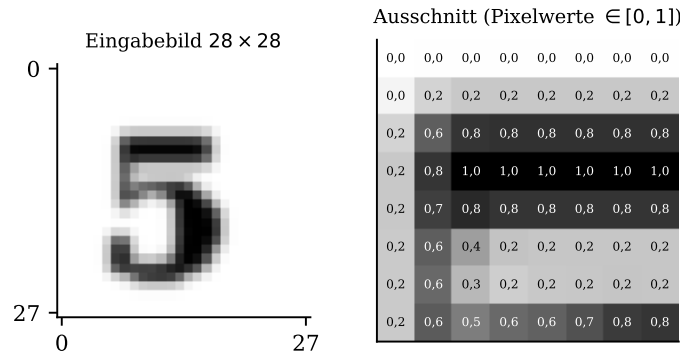


Abbildung 4: MNIST-artiges Eingabebild (schematisch): 28×28 Pixel mit Intensitäten in $[0, 1]$. Ein MLP sieht nur den Vektor der 784 Pixelwerte.

2 Vom Neuron zum Perzeptron

2.1 Das McCulloch-Pitts-Neuron

Als Modell eines biologischen Neurons betrachten wir eine Einheit k , die „feuert“ ($h_k = 1$), wenn die gewichtete Summe ihrer Eingänge eine Schwelle überschreitet:

$$h_k(\mathbf{x}) = H(\mathbf{w}_k^T \mathbf{x} - b_k), \quad H(a) = \mathbb{I}(a > 0). \quad (4)$$

Die Gewichte $\mathbf{w}_k \in \mathbb{R}^D$ entsprechen den Dendriten (Stärke der eingehenden Verbindungen), die Schwelle b_k dem Auslösen eines Aktionspotenzials. Das ist das *McCulloch-Pitts-Modell* (1943). Die Analogie endet allerdings schnell: Echte Gehirne nutzen kein Backpropagation, haben massive Rückkopplungen und Neuronen mit komplexer räumlich-zeitlicher Dynamik. Für unsere Zwecke wird das Neuron als eine vereinfachte *Rechenvorschrift* betrachtet.

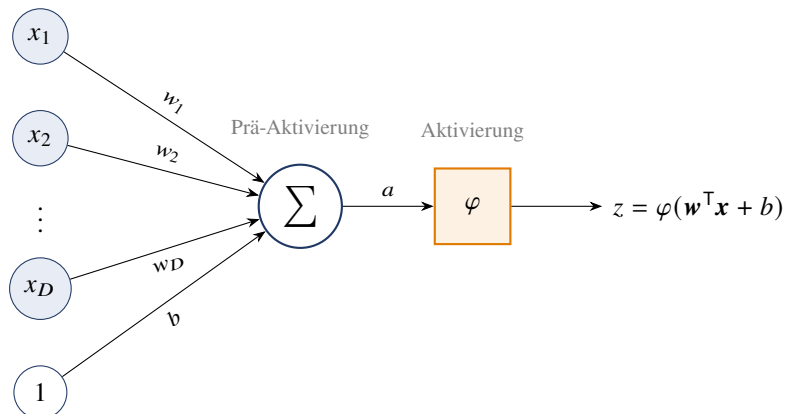


Abbildung 5: Ein künstliches Neuron: gewichtete Summe (Prä-Aktivierung $a = \mathbf{w}^T \mathbf{x} + b$) gefolgt von einer Nichtlinearität φ . Beim Perzeptron ist φ eine Sprungfunktion; bei modernen Netzen eine differenzierbare Funktion wie ReLU. Der Bias b ist die Gewichtung eines konstanten Eingangs 1.

2.2 Das Perzeptron und seine Grenze: XOR

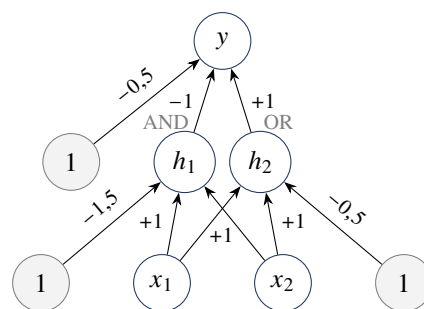
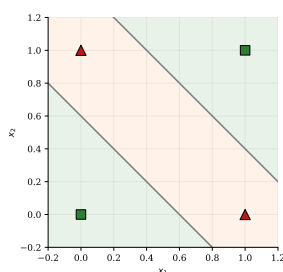
Das Perzeptron (Rosenblatt, 1958) ist genau die Einheit aus Gl. (4), mit einer Lernregel, die Gewichte bei Fehlklassifikation anpasst. Man kann es als deterministische Variante der logistischen Regression lesen: Statt

$\sigma(\mathbf{w}^\top \mathbf{x} + b)$ als Wahrscheinlichkeit auszugeben, wird hart auf $\{0, 1\}$ geschnitten. Die Entscheidungsgrenze $\mathbf{w}^\top \mathbf{x} + b = 0$ ist eine Hyperebene – ein Perzeptron kann also nur *linear separierbare* Probleme lösen.

Beispiel 2.1: Das XOR-Problem

Gesucht ist $y = x_1 \oplus x_2$ für binäre Eingaben. Die Wahrheitstafel und die Lage der Punkte zeigt Abb. 6(a): Die beiden Klassen liegen „über Kreuz“, keine Gerade trennt sie. Minsky & Papert (1969) nutzten dieses und ähnliche Beispiele, um die Grenzen des Perzeptrons zu belegen – was die Forschung an neuronalen Netzen für Jahre bremste.

Die Lösung ist das *Stapeln* von Perzeptronen (Abb. 6(b)): Zwei verdeckte Einheiten berechnen $h_1 = x_1 \wedge x_2$ (AND, Bias $-1,5$) und $h_2 = x_1 \vee x_2$ (OR, Bias $-0,5$); die Ausgabe berechnet $y = \bar{h}_1 \wedge h_2$ (Gewichte $-1, +1$, Bias $-0,5$). Nachrechnen für $\mathbf{x} = (1, 1)$: $h_1 = H(1 + 1 - 1,5) = 1$, $h_2 = H(1 + 1 - 0,5) = 1$, $y = H(-1 + 1 - 0,5) = 0$. ✓ Im transformierten Raum (h_1, h_2) ist das Problem linear separierbar: Die verdeckte Schicht hat die Merkmale so umgebaut, dass ein lineares Modell ausreicht.



(a) Nicht linear separierbar; das Band (blau) wird erst durch zwei Hyperebenen ausgeschnitten.

(b) Zweischichtiges Netz mit handgesetzten Gewichten, Berechnung von XOR.

Abbildung 6: Das XOR-Problem und seine Lösung durch eine verdeckte Schicht (nach Murphy 2022, Abb. 13.1).

Verallgemeinert man das Beispiel, lässt sich zeigen, dass ein MLP jede logische Funktion darstellen kann. Der entscheidende Schritt ist aber ein anderer: Wir wollen die Gewichte *nicht* von Hand setzen, sondern aus Daten lernen. Dafür ist die Sprungfunktion ungeeignet da ihre Ableitung fast überall null ist und sie keinerlei Gradientensignal liefert..l,

3 Das Multilayer-Perzeptron

3.1 Architektur und Vorwärtsrechnung

Definition 3.1: Multilayer-Perzeptron (MLP)

Ein MLP mit L Schichten ist die Funktion $f(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{z}_L$, definiert durch die Rekursion

$$\mathbf{z}_0 = \mathbf{x}, \quad \mathbf{a}_l = \mathbf{W}_l \mathbf{z}_{l-1} + \mathbf{b}_l, \quad \mathbf{z}_l = \varphi_l(\mathbf{a}_l), \quad l = 1, \dots, L, \quad (5)$$

mit Gewichtsmatrizen $\mathbf{W}_l \in \mathbb{R}^{K_l \times K_{l-1}}$, Bias-Vektoren $\mathbf{b}_l \in \mathbb{R}^{K_l}$ und elementweisen, differenzierbaren Aktivierungsfunktionen φ_l . Die Schichten $1, \dots, L-1$ heißen *verdeckt* (hidden), ihre Ausgaben sind in den Trainingsdaten nicht beobachtet. In Skalarschreibweise: $z_{kl} = \varphi_l(b_{kl} + \sum_{j=1}^{K_{l-1}} w_{jkl} z_{j,l-1})$.

Beispiel 3.2: MLP für MNIST – Parameter zählen

Ein MLP mit Eingabe $\mathbb{R}^{784}$, zwei verdeckten Schichten à 128 Einheiten (ReLU) und einer Softmax-

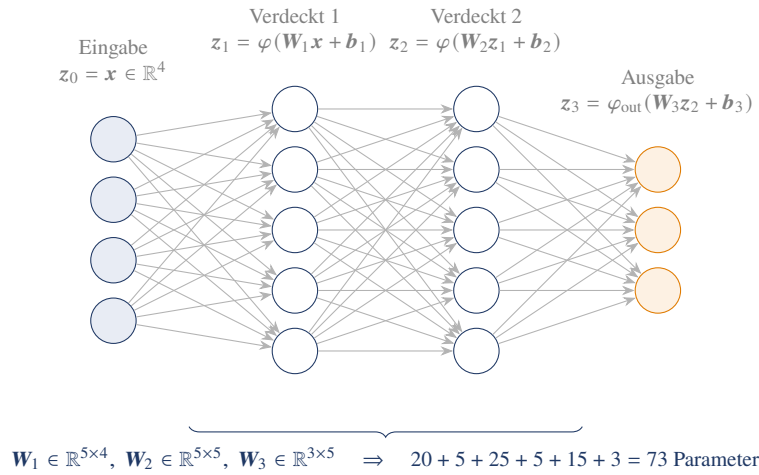


Abbildung 7: Vollverbundenes MLP mit zwei verdeckten Schichten. Jede Kante trägt ein Gewicht; jede Einheit einen Bias. Für eine Schicht $\mathbb{R}^{K_{l-1}} \rightarrow \mathbb{R}^{K_l}$ gilt $\#Parameter = K_l(K_{l-1} + 1)$.

Ausgabe mit 10 Klassen hat

$$\underbrace{(784 + 1) \cdot 128}_{100.480} + \underbrace{(128 + 1) \cdot 128}_{16.512} + \underbrace{(128 + 1) \cdot 10}_{1.290} = 113.674 \text{ Parameter.}$$

Bereits nach zwei Epochen erreicht dieses Modell rund 97,1 % Testgenauigkeit auf MNIST (Murphy 2022, Tab. 13.2). Die erste Schicht enthält bereits 85 % aller Parameter. Das ist ein typisches Muster bei hochdimensionalen Eingangsdaten. Genau hier zeigt sich der Vorteil von CNNs: Durch das Teilen von Gewichten (Weight-Sharing) sparen sie im Vergleich enorm viele Parameter ein.

3.2 Aktivierungsfunktionen

Warum überhaupt eine Nichtlinearität? Wählt man $\varphi_l(a) = c_l a$ linear, kollabiert das ganze Netz zu einem linearen Modell:

$$f(\mathbf{x}) = \mathbf{W}_L c_L (\mathbf{W}_{L-1} c_{L-1} (\cdots \mathbf{W}_1 \mathbf{x})) \propto \mathbf{W}_L \mathbf{W}_{L-1} \cdots \mathbf{W}_1 \mathbf{x} = \mathbf{W}' \mathbf{x}. \quad (6)$$

Tiefe ohne Nichtlinearität bringt also nichts. Tabelle 2 und Abb. 8 zeigen die gängigen Aktivierungsfunktionen.

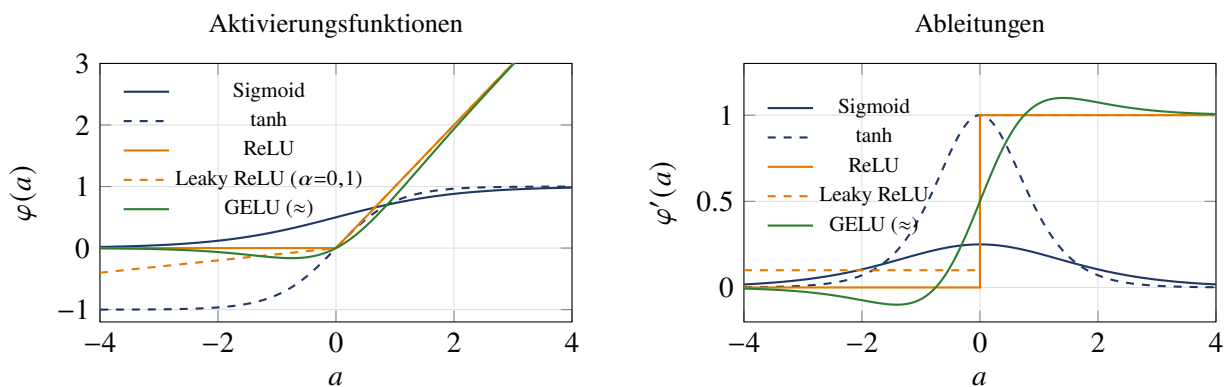


Abbildung 8: Links: Aktivierungsfunktionen. Rechts: ihre Ableitungen – entscheidend für Backpropagation. Sigmoid und tanh haben nur in einem schmalen Bereich um 0 nennenswerte Steigung; ReLU-Varianten leiten für $a > 0$ den Gradienten unverändert weiter.

Name	Definition	Wertebereich	Ableitung $\varphi'(a)$	Bemerkung
Sigmoid	$\sigma(a) = \frac{1}{1 + e^{-a}}$	$(0, 1)$	$\sigma(a)(1 - \sigma(a))$	sättigt; max. Steigung 0,25 $\Rightarrow$ vanishing gradients
tanh	$\tanh(a) = 2\sigma(2a) - 1$	$(-1, 1)$	$1 - \tanh^2(a)$	sättigt; nullzentriert, daher meist besser als Sigmoid
Softplus	$\sigma_+(a) = \log(1 + e^a)$	$(0, \infty)$	$\sigma(a)$	glatte ReLU-Variante
ReLU	$\max(a, 0)$	$[0, \infty)$	$\mathbb{I}(a > 0)$	Standard ; nicht sättigend für $a > 0$; „dead ReLU“ für $a < 0$ kein Absterben; $\alpha \approx 0,01$; lernbar $\Rightarrow$ PReLU
Leaky ReLU	$\max(a, 0) + \alpha \min(a, 0)$	$\mathbb{R}$	1 bzw. α	glatt; SELU-Variante ist selbstnormalisierend
ELU	$\max(a, 0) + \min(\alpha(e^a - 1), 0)$	$(-\alpha, \infty)$	1 bzw. αe^a	
Swish/SiLU	$a \sigma(a)$	$\mathbb{R}$	$\sigma(a)(1 + a(1 - \sigma(a)))$	per Suche gefunden; nicht monoton
GELU	$a \Phi(a)$	$\mathbb{R}$	$\Phi(a) + a \phi(a)$	„weiche“ ReLU; Standard in Transformern; $\approx a \sigma(1,702a)$

Tabelle 2: Gängige Aktivierungsfunktionen (Φ, ϕ : Verteilungs- bzw. Dichtefunktion der Standardnormalverteilung). Nach Murphy (2022), Tab. 13.4.

Merke

Sättigung ist das Problem. Für $|a| \gg 0$ ist $\sigma'(a) \approx 0$. Ein Gradientensignal, das durch viele gesättigte Sigmoid-Schichten zurückläuft, wird in jeder Schicht um einen Faktor $\leq 0,25$ gedämpft und verschwindet exponentiell mit der Tiefe (Abschn. 7). Nicht sättigende Aktivierungen wie ReLU sind einer der Schlüssel zum Training tiefer Netze.

In der Praxis

Empfehlung für verdeckte Schichten: **ReLU als Default**. Bei Anzeichen für „tote“ Einheiten (viele Aktivierungen konstant 0) auf Leaky ReLU oder ELU wechseln. In Transformer-Architekturen ist GELU üblich. Sigmoid und tanh gehören heute in die *Ausgabe*-Schicht bzw. in Gating-Mechanismen (LSTM), nicht in tiefe verdeckte Stapel.

3.3 Universelle Approximation und die Rolle der Tiefe

Satz 3.3: Universeller Approximationssatz (Hornik et al. 1989, Cybenko 1989)

Ein MLP mit *einer* verdeckten Schicht und einer nicht-polynomialen Aktivierungsfunktion kann jede stetige Funktion auf einer kompakten Teilmenge des $\mathbb{R}^D$ beliebig genau approximieren – sofern genügend verdeckte Einheiten zur Verfügung stehen.

Intuition: Jede verdeckte ReLU-Einheit definiert eine Halbebene; hinreichend viele davon „zerlegen“ den Eingaberaum in stückweise lineare Regionen, denen beliebige Ausgaben zugeordnet werden können. Der Satz sagt allerdings nichts darüber, *wie viele* Einheiten nötig sind (im Worst Case exponentiell viele zu D) und ob die Parameter per Gradientenabstieg *gefunden* werden. Beides spricht für Tiefe:

- **Kompositionalität.** Spätere Schichten nutzen die Merkmale früherer Schichten. Ein Klassifikator für DNA-Sequenzen mit dem Muster `*AA??CGCG??AA*` lernt leichter, wenn Schicht 1 die Motive AA und CG erkennt und Schicht 2 sie linear kombiniert (analog zur XOR-Lösung).
- **Effizienz.** Für viele Funktionsklassen wächst die nötige Breite eines flachen Netzes exponentiell, die Tiefe eines tiefen Netzes nur linear (Montúfar et al. 2014; Goodfellow et al. 2016, Abschn. 6.4.1). Empirisch generalisieren tiefere Netze bei gleicher Parameterzahl oft besser.

3.4 Ausgabeschichten und Loss-Funktionen

Die Wahl von Ausgabeaktivierung und Loss ist keine Geschmacksfrage, sondern folgt aus dem *Maximum-Likelihood-Prinzip*: Das Netz parametrisiert eine bedingte Verteilung $p(y | \mathbf{x}; \boldsymbol{\theta})$, und wir minimieren die negativ logarithmierte Plausibilität (Negative Log-Likelihood, NLL)

$$\mathcal{L}(\boldsymbol{\theta}) = -\log p(\mathcal{D} | \boldsymbol{\theta}) = -\sum_{n=1}^N \log p(y_n | \mathbf{x}_n; \boldsymbol{\theta}). \quad (7)$$

Je nach Ausgabeverteilung ergeben sich die bekannten Loss-Funktionen:

Aufgabe	Verteilung	Aktivierung	Loss (pro Beispiel)	PyTorch
Regression	$\mathcal{N}(y \hat{y}, \sigma^2)$	linear (Identität)	$\frac{1}{2}(y - \hat{y})^2$ (MSE)	<code>nn.MSELoss</code>
Binär	$\text{Ber}(y \sigma(a))$	Sigmoid	$-[y \log \hat{p} + (1 - y) \log(1 - \hat{p})]$	<code>nn.BCEWithLogitsLoss</code>
C Klassen	$\text{Cat}(y \text{softmax}(\mathbf{a}))$	Softmax	$-\sum_c y_c \log \hat{p}_c$ (Cross-Entropy)	<code>nn.CrossEntropyLoss</code>

Für die Klassifikation mit C Klassen bildet die Softmax-Funktion die Logits $\mathbf{a} \in \mathbb{R}^C$ auf eine Wahrscheinlichkeitsverteilung ab:

$$\hat{p}_c = \text{softmax}(\mathbf{a})_c = \frac{e^{a_c}}{\sum_{c'=1}^C e^{a_{c'}}}, \quad \mathcal{L} = -\log \hat{p}_{c^*} = \log \sum_j e^{a_j} - a_{c^*}, \quad (8)$$

wobei c^* die wahre Klasse ist. Die zweite Form („log-sum-exp minus Logit“) ist numerisch stabil und zeigt, warum Cross-Entropy die richtige Wahl ist: Die Ableitung nach den Logits ist einfach $\partial \mathcal{L} / \partial a_i = \hat{p}_i - y_i$ (Herleitung in Abschn. 4.4.1). Selbst wenn die Ausgabe stark sättigt (ein $\hat{p}_c \rightarrow 1$ für die falsche Klasse), bleibt der Gradient groß da die Sättigung der Softmax-Aktivierungsfunktion durch den Logarithmus im Loss „aufgehoben“ wird. Mit MSE auf Softmax-Ausgaben wäre das nicht der Fall. Das Training würde in falschen, aber sicheren Vorhersagen stecken bleiben.

In der Praxis

Nutzen Sie `nn.CrossEntropyLoss` in PyTorch immer ohne vorherige `nn.Softmax`-Schicht. Die Funktion erwartet als Eingabe die rohen, unnormierten Vorhersagen Ihres Modells (*Logits*) und keine fertigen Wahrscheinlichkeiten. PyTorch kombiniert die mathematischen Schritte *Softmax* und *Log* intern so, dass Berechnungsfehler und Instabilitäten verhindert werden. Wenn Sie fälschlicherweise eine *Softmax*-Schicht im Modell vorschalten, funktioniert das Training zwar ohne Fehlermeldungen, aber Ihr Modell lernt dadurch deutlich schlechter.

3.5 Das MLP in PyTorch

Die Definition aus Gl. (5) übersetzt sich fast eins zu eins in Code. `nn.Linear` implementiert $\mathbf{W}\mathbf{z} + \mathbf{b}$ (mit passender Initialisierung, Abschn. 5.6), `nn.ReLU` die Aktivierung.

Listing 1: MLP für MNIST in PyTorch. Die Ausgabe sind Logits; Softmax ist Teil des Loss.

```
import torch, torch.nn as nn

class MLP(nn.Module):
    def __init__(self, d_in=784, d_hidden=128, n_classes=10):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(), # (B, 1, 28, 28) -> (B, 784)
            nn.Linear(d_in, d_hidden), nn.ReLU(),
            nn.Linear(d_hidden, d_hidden), nn.ReLU(),
            nn.Linear(d_hidden, n_classes), # Logits, kein Softmax!
        )
```

```
def forward(self, x):
    return self.net(x)

model = MLP()
n_params = sum(p.numel() for p in model.parameters())
print(n_params)    # 118282
```

4 Gradientenbasiertes Lernen und Backpropagation

4.1 Lernen als Optimierungsproblem

Training heißt: Finde $\theta^* = \arg \min_{\theta} \mathcal{L}(\theta)$. Da $\mathcal{L}$ für neuronale Netze nicht konvex ist, gibt es keine geschlossene Lösung und keine Garantie auf das globale Minimum. Der praktikable Weg ist *Gradientenabstieg*: Wiederhole

$$\theta \leftarrow \theta - \epsilon \nabla_{\theta} \mathcal{L}(\theta), \quad (9)$$

mit Lernrate $\epsilon > 0$. Der Gradient zeigt in Richtung des steilsten Anstiegs; ein Schritt gegen ihn senkt $\mathcal{L}$ für hinreichend kleines ϵ . Alles hängt also daran, $\nabla_{\theta} \mathcal{L}$ für ein Netz mit Millionen Parametern *effizient* zu berechnen. Genau das leistet Backpropagation.

Merke

Backpropagation ist *kein* Lernalgorithmus, sondern ein Verfahren zur Gradientenberechnung: die systematische Anwendung der Kettenregel auf einem Berechnungsgraphen, in umgekehrter Reihenfolge der Auswertung. Das Lernen selbst übernimmt der Optimierer (Abschn. 5).

4.2 Berechnungsgraphen und die Kettenregel

Jedes Netz – und jede Loss-Funktion – ist eine Komposition elementarer Operationen. Das lässt sich als gerichteter azyklischer Graph (DAG) darstellen, in dem jeder Knoten eine differenzierbare Funktion seiner Eltern ist. Abbildung 9 zeigt ein kleines Beispiel.

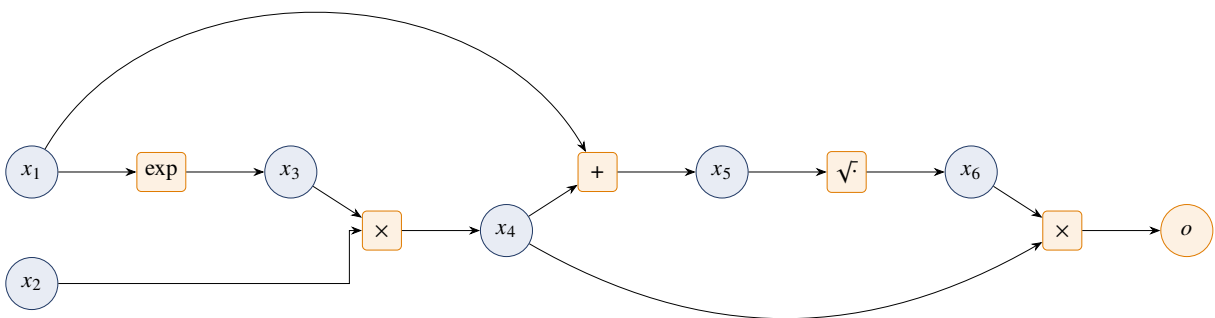


Abbildung 9: Berechnungsgraph für $f(x_1, x_2) = x_2 e^{x_1} \sqrt{x_1 + x_2 e^{x_1}}$ mit Zwischengrößen $x_3 = e^{x_1}$, $x_4 = x_2 x_3$, $x_5 = x_1 + x_4$, $x_6 = \sqrt{x_5}$, $o = x_4 x_6$ (nach Murphy 2022, Abb. 13.11). Knoten x_4 beeinflusst die Ausgabe über zwei Pfade – im Rückwärtsschritt müssen die Beiträge summiert werden.

Für eine Kette $f = f_3 \circ f_2 \circ f_1$ mit $\mathbf{x}_2 = f_1(\mathbf{x})$, $\mathbf{x}_3 = f_2(\mathbf{x}_2)$ usw. liefert die Kettenregel die Jacobi-Matrix als Produkt:

$$J_f(\mathbf{x}) = \frac{\partial \mathbf{o}}{\partial \mathbf{x}} = \underbrace{\frac{\partial \mathbf{o}}{\partial \mathbf{x}_3}}_{m \times m_2} \underbrace{\frac{\partial \mathbf{x}_3}{\partial \mathbf{x}_2}}_{m_2 \times m_1} \underbrace{\frac{\partial \mathbf{x}_2}{\partial \mathbf{x}}}_{m_1 \times n} = J_{f_3} J_{f_2} J_{f_1}. \quad (10)$$

Bei der Matrixmultiplikation kommt es auf die richtige Reihenfolge an: Das Ergebnis bleibt zwar immer gleich, der Rechenaufwand unterscheidet sich teils extrem.

4.3 Vorwärts- oder Rückwärtsmodus?

Zwei Strategien, das Produkt in Gl. (10) auszuwerten:

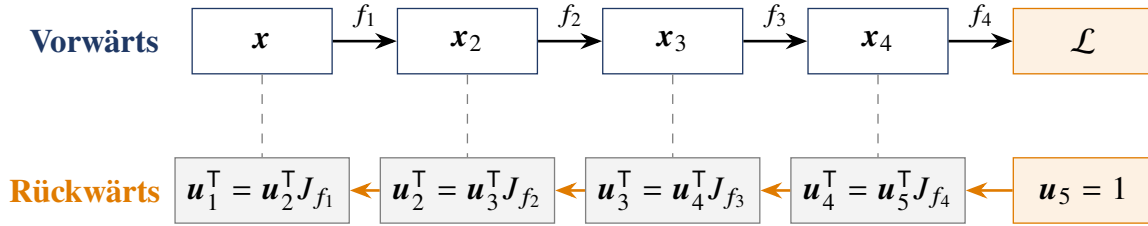
- **Vorwärtsmodus** (rechts nach links): Man propagiert einen Spaltenvektor $\mathbf{v}$ durch *Jacobi-Vektor-Produkte* (JVP) $J_{f_k} \mathbf{v}$, parallel zur Vorwärtsrechnung. Mit $\mathbf{v} = \mathbf{e}_j$ erhält man die j -te Spalte von J_f . Für die volle Jacobi-Matrix sind n Durchläufe nötig (n = Eingabedimension). Vorteil dieses Modus ist, dass für jeden Zwischenschritt durch die **Matrix · Vektor** Multiplikation immer ein Vektor berechnet wird und damit lediglich Kosten von $O(N^2)$ entstehen.

$$J_f \cdot \mathbf{v} = (J_{f_3} \cdot (J_{f_2} \cdot (J_{f_1} \cdot \mathbf{v}))) \quad (11)$$

- **Rückwärtsmodus** (links nach rechts): Man propagiert einen Zeilenvektor $\mathbf{u}^\top$ durch *Vektor-Jacobi-Produkte* (VJP) $\mathbf{u}^\top J_{f_k}$, in umgekehrter Reihenfolge. Mit $\mathbf{u} = \mathbf{e}_i$ erhält man die i -te Zeile. Für die volle Matrix sind m Durchläufe nötig (m = Ausgabedimension). Dieser Modus eignet sich besonders für viele Ein- bei wenigen Ausgaben da die Ableitungen für beliebig viele Eingabevariablen in einem einzigen Durchlauf berechnet werden, was ihn extrem zeiteffizient für das Training großer Modelle macht.

$$\mathbf{u}^\top \cdot J_f = (((\mathbf{u}^\top \cdot J_{f_3}) \cdot J_{f_2}) \cdot J_{f_1}) \quad (12)$$

Beim Training ist die Ausgabe ein *Skalar* (der Loss, $m = 1$) und die Eingabe sind Millionen Parameter ($n \gg 1$). Der Rückwärtsmodus liefert den vollständigen Gradienten in *einem* Durchlauf, dessen Kosten in derselben Größenordnung liegen wie ein Vorwärtsdurchlauf. Der Vorwärtsmodus bräuchte n Durchläufe. Backpropagation verdankt seinen Namen der *Fehlerrückführung*: Das Verfahren berechnet die Anpassungen für das Netzwerk, indem es den *Loss von hinten nach vorne ableitet*.



Gestrichelt: Die Jacobi-Matrizen J_{f_k} hängen von den im Vorwärtsdurchlauf gespeicherten $\mathbf{x}_k$ ab. Daher der Speicherbedarf des Trainings: Alle Aktivierungen werden bis zum Rückwärtsschritt gehalten.

Abbildung 10: Rückwärtsmodus-Autodiff auf einer Kette. Der Adjungierte $\mathbf{u}_k = \partial \mathcal{L} / \partial \mathbf{x}_k$ wird von der Ausgabe zur Eingabe durch VJPs propagiert.

4.4 Backpropagation für ein MLP

Nun erhalten die Netzwerkschichten Ihre Eingabe ($\mathbf{x}_k$), Parameter (θ_k) und eine Ausgabe ($\mathbf{x}_{k+1}$): $\mathbf{x}_{k+1} = f_k(\mathbf{x}_k, \theta_k)$. Für die Optimierung nutzen wir das *Fehlersignal* ($\mathbf{u}_{k+1}$), auch Adjungierte genannt. Dieses Signal zeigt, wie stark sich der Gesamtfehler ($\mathcal{L}$) ändert, wenn sich die Ausgabe dieser Schicht ändern. Der Adjungierte $\mathbf{u}_{k+1}^\top = \partial \mathcal{L} / \partial \mathbf{x}_{k+1}$ liefert den Gradienten nach den Parametern der Schicht und den Adjungierten für die vorherige Schicht:

$$\underbrace{\mathbf{g}_k}_{\nabla_{\theta_k} \mathcal{L}} = \mathbf{u}_{k+1}^\top \frac{\partial f_k(\mathbf{x}_k, \theta_k)}{\partial \theta_k}, \quad \mathbf{u}_k^\top = \mathbf{u}_{k+1}^\top \frac{\partial f_k(\mathbf{x}_k, \theta_k)}{\partial \mathbf{x}_k}. \quad (13)$$

Es müssen nun noch die VJPs der elementaren Schichttypen definiert werden. Ein MLP lässt sich durch drei Standardfälle vollständig beschreiben.

Voraussetzung: Eingabe $\mathbf{x}$, Parameter $\{\boldsymbol{\theta}_k\}$, Schichtfunktionen f_k
 // Vorwärtsthroughlauf: Aktivierungen speichern
 $\mathbf{x}_1 \leftarrow \mathbf{x}$
for $k = 1, \dots, K$ **do**
 $\mathbf{x}_{k+1} \leftarrow f_k(\mathbf{x}_k, \boldsymbol{\theta}_k)$
end
 // Rückwärtsthroughlauf: Adjungierte propagieren
 $\mathbf{u}_{K+1} \leftarrow 1$
for $k = K, \dots, 1$ **do**
 $\mathbf{g}_k \leftarrow \mathbf{u}_{k+1}^\top \partial f_k(\mathbf{x}_k, \boldsymbol{\theta}_k) / \partial \boldsymbol{\theta}_k$ // Gradient nach Parametern
 $\mathbf{u}_k^\top \leftarrow \mathbf{u}_{k+1}^\top \partial f_k(\mathbf{x}_k, \boldsymbol{\theta}_k) / \partial \mathbf{x}_k$ // Adjungierter für Schicht $k-1$
end
return $\mathcal{L} = \mathbf{x}_{K+1}$, $\nabla_{\mathbf{x}} \mathcal{L} = \mathbf{u}_1$, $\{\nabla_{\boldsymbol{\theta}_k} \mathcal{L} = \mathbf{g}_k\}_{k=1}^K$
Algorithm 1: Backpropagation für ein MLP mit K Schichten (Murphy 2022, Alg. 7)

4.4.1 VJPs der Standardschichten

Cross-Entropy-Schicht. Für $\mathcal{L} = -\sum_c y_c \log \hat{p}_c$ mit $\hat{\mathbf{p}} = \text{softmax}(\mathbf{a})$ und wahrer Klasse c^* gilt $\mathcal{L} = \log \sum_j e^{a_j} - a_{c^*}$, also

$$\frac{\partial \mathcal{L}}{\partial a_i} = \frac{e^{a_i}}{\sum_j e^{a_j}} - \mathbb{I}(i = c^*) = \hat{p}_i - y_i \implies \frac{\partial \mathcal{L}}{\partial \mathbf{a}} = (\hat{\mathbf{p}} - \mathbf{y})^\top \in \mathbb{R}^{1 \times C}. \quad (14)$$

Das ist bemerkenswert einfach: Der Fehler „Vorhersage minus Ziel“ startet die Rückwärtsrechnung.

Elementweise Nichtlinearität. Für $\mathbf{z} = \varphi(\mathbf{a})$ ist die Jacobi-Matrix diagonal, $\mathbf{J} = \text{diag}(\varphi'(\mathbf{a}))$, und das VJP ein elementweises Produkt: $\mathbf{u}^\top \mathbf{J} = (\mathbf{u} \odot \varphi'(\mathbf{a}))^\top$. Für ReLU gilt $\varphi'(a) = \mathbb{I}(a > 0)$: Der Gradient wird dort durchgelassen, wo die Einheit aktiv war, und sonst abgeschnitten.

Lineare Schicht. Für $\mathbf{z} = \mathbf{W}\mathbf{x}$ mit $\mathbf{W} \in \mathbb{R}^{m \times n}$ gilt $\partial \mathbf{z} / \partial \mathbf{x} = \mathbf{W}$, also

$$\mathbf{u}^\top \frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \mathbf{u}^\top \mathbf{W} \in \mathbb{R}^{1 \times n} \quad \text{und} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{u} \mathbf{x}^\top \in \mathbb{R}^{m \times n}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \mathbf{u}. \quad (15)$$

Die zweite Formel folgt aus $\partial z_k / \partial W_{ij} = x_j \mathbb{I}(k = i)$, also $\sum_k u_k \partial z_k / \partial W_{ij} = u_i x_j$. Der Gradient einer Gewichtsmatrix ist das *äußere Produkt* aus rücklaufendem Fehler und Eingang der Schicht – die Hebb-Regel „what fires together, wires together“ in gradientenbasierter Form.

Beispiel 4.1: Vollständige Herleitung für ein einschichtiges MLP

Betrachte $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}_1 \in \mathbb{R}^K$, $\mathbf{h} = \text{ReLU}(\mathbf{z})$, $\mathbf{a} = \mathbf{V}\mathbf{h} + \mathbf{b}_2 \in \mathbb{R}^C$, $\mathcal{L} = \text{CE}(\mathbf{y}, \text{softmax}(\mathbf{a}))$. Rückwärts, Schicht für Schicht:

$$\begin{aligned} \mathbf{u}_2 &= \nabla_{\mathbf{a}} \mathcal{L} = \hat{\mathbf{p}} - \mathbf{y} \in \mathbb{R}^C && \text{(Cross-Entropy)} \\ \nabla_{\mathbf{V}} \mathcal{L} &= \mathbf{u}_2 \mathbf{h}^\top \in \mathbb{R}^{C \times K}, \quad \nabla_{\mathbf{b}_2} \mathcal{L} = \mathbf{u}_2 && \text{(lineare Schicht)} \\ \mathbf{u}_1 &= \nabla_{\mathbf{z}} \mathcal{L} = (\mathbf{V}^\top \mathbf{u}_2) \odot H(\mathbf{z}) \in \mathbb{R}^K && \text{(linear, dann ReLU)} \\ \nabla_{\mathbf{W}} \mathcal{L} &= \mathbf{u}_1 \mathbf{x}^\top \in \mathbb{R}^{K \times D}, \quad \nabla_{\mathbf{b}_1} \mathcal{L} = \mathbf{u}_1 && \text{(lineare Schicht)} \\ \nabla_{\mathbf{x}} \mathcal{L} &= \mathbf{W}^\top \mathbf{u}_1 \in \mathbb{R}^D && \text{(optional; z. B. für Adversarial Examples)} \end{aligned}$$

Jeder Schritt ist eine Matrix-Vektor-Multiplikation oder ein äußeres Produkt: Der Rückwärtsthroughlauf kostet etwa das Doppelte des Vorwärtsthroughlaufs. (Entspricht Übung 13.1 in Murphy 2022; im Lab verifizieren Sie diese Formeln optional per NumPy und finiten Differenzen.)

4.5 Allgemeine Graphen

Ist der Graph keine Kette, hat ein Knoten j mehrere Kinder, und der Adjungierte summiert über alle Pfade:

$$\frac{\partial o}{\partial \mathbf{x}_j} = \sum_{k \in \text{children}(j)} \frac{\partial o}{\partial \mathbf{x}_k} \frac{\partial \mathbf{x}_k}{\partial \mathbf{x}_j}. \quad (16)$$

Der Trick besteht darin, die Knoten des Graphen in *umgekehrter topologischer Reihenfolge* abzuarbeiten, also von hinten (Ausgabe) nach vorne (Eingabe). Dadurch ist beim Bearbeiten eines Knotens j die Ableitung $\partial o / \partial \mathbf{x}_k$ für alle seine "Kinder" (die nachfolgenden Knoten, welche von j abhängen) bereits fertig berechnet und nichts muss doppelt berechnet werden. Ein Beispiel dafür, wie das bei mehreren Pfaden in Abb. 9 funktioniert: Wenn ein Knoten x_4 über zwei verschiedene Wege zum Ergebnis beiträgt – einmal über x_5 und einmal über x_7 – werden schlicht die Beiträge beider Pfade addiert:

$$\frac{\partial o}{\partial x_4} = \frac{\partial o}{\partial x_5} \frac{\partial x_5}{\partial x_4} + \frac{\partial o}{\partial x_7} \frac{\partial x_7}{\partial x_4} \quad (17)$$

Die Regel Beiträge über verschiedene Pfade aufzusummieren ist kein Sonderfall, sondern taucht in vielen bekannten Architekturen wieder auf: Skip-Connections in ResNets (die Information über mehrere Wege gleichzeitig weiterreichen), Attention-Mechanismen und Weight Sharing in CNNs (wo dasselbe Gewicht an mehreren Stellen im Netz verwendet wird) funktionieren alle nach genau diesem Prinzip. In der Frage wie dieser Graph entsteht unterscheiden sich die Frameworks:

- **Statisch** (z. B. TensorFlow 1): Der komplette Rechengraph wird vorab einmal festgelegt, bevor überhaupt Daten hindurchlaufen.
- **Dynamisch** (PyTorch, JAX, TensorFlow Eager): Der Graph entsteht erst zur Laufzeit, indem das Framework mitschreibt ("tracing"), welche Rechenschritte tatsächlich ausgeführt werden.

Der Vorteil der dynamischen Variante: Das Modell kann seinen Kontrollfluss von den Daten abhängig machen. Beispielsweise können unterschiedlich viele Schleifendurchläufe oder andere Verzweigungen durchlaufen, je nachdem, welche Eingabe vorliegt. Bei einem statischen Graphen ist dies nicht möglich, da die Struktur schon im Vorhinein feststeht.

4.6 Autograd in PyTorch

Sie werden Backpropagation nie von Hand implementieren müssen – aber Sie sollten wissen, was die drei Zeilen tun, die in jedem Trainingsloop stehen:

Listing 2: Ein Trainingsschritt in PyTorch. `backward()` führt algorithm 1 auf dem dynamisch aufgezeichneten Graphen aus.

```
loss_fn    = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

for x, y in train_loader:
    logits = model(x)                # Minibatch (B,1,28,28), (B,)
    # Vorwaertsdurchlauf, Graph wird
    # aufgezeichnet
    loss    = loss_fn(logits, y)      # skalarer Loss (Mittel ueber Batch)
    optimizer.zero_grad()            # alte Gradienten loeschen (sie
    # akkumulieren!)
    loss.backward()                  # Rueckwaertsdurchlauf: p.grad fuer
    # alle Parameter
    optimizer.step()                 # Parameter-Update gemaess
    # Optimierer
```

Jeder `torch.Tensor` mit `requires_grad=True` merkt sich die Operation, die ihn erzeugt hat (`grad_fn`). `loss.backward()` traversiert diesen Graphen rückwärts und schreibt die Gradienten in `p.grad`. Gradienten werden *akkumuliert*, deshalb `zero_grad()` vor jedem Schritt. Mit `torch.no_grad()` schalten Sie die Aufzeichnung ab. Dies ist Pflicht bei der Evaluation, um Speicher zu sparen.

5 Optimierung: SGD, Momentum und adaptive Verfahren

5.1 Lernen ist nicht reine Optimierung

Eigentlich wollen wir das *Risiko* $\mathbb{E}_{(\mathbf{x}, y) \sim p_{\text{data}}} [\ell(f(\mathbf{x}; \boldsymbol{\theta}), y)]$ minimieren – den erwarteten Verlust auf *neuen* Daten. Da p_{data} unbekannt ist, minimieren wir das *empirische Risiko* auf den N Trainingsbeispielen. Daraus ergeben sich zwei Konsequenzen:

1. Der Trainingsverlust ist nur ein Stellvertreter. Ein Optimierer, der ihn perfekt auf null drückt, kann das Ziel verfehlen (Overfitting, Abschn. 6). Deshalb wird oft *vor* Erreichen eines Minimums gestoppt.
2. Der Gradient über alle N Beispiele ist ein Mittelwert. Ein Mittelwert über eine zufällige Teilmenge (*Minibatch*) von $m \ll N$ Beispielen ist ein erwartungstreuer Schätzer mit Standardfehler $\propto \sigma / \sqrt{m}$. Jede zusätzliche Beispiel-Verdopplung bringt immer weniger zusätzliche Genauigkeit. Sehr große Batches sind darum eine ineffiziente Investition. Eine Verhundertfachung der Batch-Größe senkt beispielsweise den Fehler um den Faktor 10 während die Rechenkosten um den Faktor 1000 steigen. Man zahlt also den zehnfachen Preis für die halbe Verbesserung.

Definition 5.1: Batch, Minibatch, Epoche

Batch-Gradientenabstieg nutzt alle N Beispiele pro Update; *stochastischer Gradientenabstieg* (SGD) im engen Sinn $m = 1$; in der Praxis *Minibatch-SGD* mit $m \in \{32, \dots, 512\}$, meist Zweierpotenzen wegen der GPU-Auslastung. Eine *Epoche* ist ein vollständiger Durchlauf durch die Trainingsdaten, also $\lceil N/m \rceil$ Updates (*Iterationen*). Die Beispiele werden vor jeder Epoche neu gemischt, damit die Minibatches unabhängige Stichproben bleiben.

5.2 Stochastischer Gradientenabstieg

Voraussetzung: Lernratenplan ϵ_k , Startparameter $\boldsymbol{\theta}$

$k \leftarrow 0$

while Abbruchkriterium nicht erfüllt **do**

Ziehe Minibatch $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$ aus den Trainingsdaten

$\hat{\mathbf{g}} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i \ell(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$

// per Backprop

$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon_k \hat{\mathbf{g}}; \quad k \leftarrow k + 1$

end

Algorithm 2: Minibatch-SGD (Goodfellow et al. 2016, Alg. 8.1)

Die Lernrate ϵ ist der wichtigste Hyperparameter. Weil der Minibatch-Gradient auch am Minimum noch rauscht, muss ϵ_k mit der Zeit sinken; hinreichend für Konvergenz ist $\sum_k \epsilon_k = \infty$ und $\sum_k \epsilon_k^2 < \infty$. Übliche Pläne: schrittweise (*step decay*), linear bis Iteration τ und dann konstant, oder Cosinus-Abfall; bei großen Modellen zusätzlich ein kurzes *Warm-up* von 0 auf ϵ_0 . Abbildung 11 zeigt die vier typischen Verhaltensweisen von Modellen während des Trainings.

5.3 Was die Optimierung tiefer Netze schwer macht

Schlechte Konditionierung

Ist die Hesse-Matrix $\mathbf{H}$ des Verlusts stark anisotrop (Konditionszahl $\gg 1$), zwingt die steilste Richtung zu kleinen Schritten, während die flache Richtung kaum vorankommt (Abb. 13, links). Ein Schritt $-\epsilon \mathbf{g}$ senkt $\mathcal{L}$ um $\epsilon \mathbf{g}^\top \mathbf{g} - \frac{1}{2} \epsilon^2 \mathbf{g}^\top \mathbf{H} \mathbf{g}$; wächst der zweite Term schneller als der erste, steigt der Verlust trotz großem Gradienten.

Lokale Minima

Neuronale Netze haben exponentiell viele äquivalente Minima (Permutation verdeckter Einheiten). Empirisch haben lokale Minima in großen Netzen einen Verlust nahe dem globalen und sind daher in der Praxis

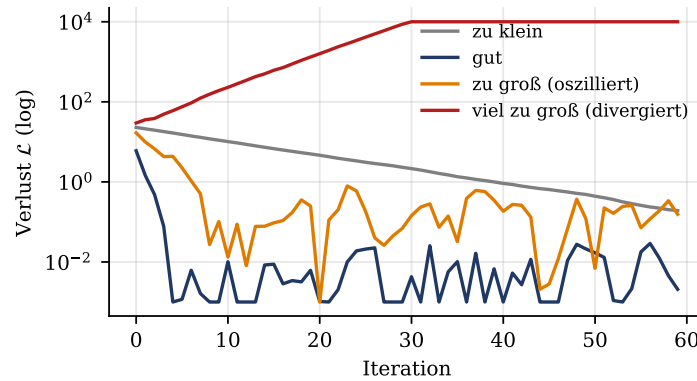


Abbildung 11: Effekt der Lernrate auf einer verrauschten quadratischen Zielfunktion ($\mathcal{L} = \theta^2$, Gradient mit Rauschen). Zu klein: langsam. Zu groß: Oszillation um das Minimum, das Rauschen dominiert. Viel zu groß: Divergenz. Eine gute Lernrate ist typischerweise die größte, die noch stabil ist – gefunden per Rastersuche über Zehnerpotenzen.

selten ein Problem.

Sattelpunkte und Plateaus

In hoher Dimension sind stationäre Punkte mit gemischter Krümmung (Abb. 12) exponentiell häufiger als Minima. Der Gradient wird dort klein, ohne dass ein Minimum vorliegt; Verfahren erster Ordnung entkommen meist durch das Rauschen, Newton-Verfahren werden dagegen *angezogen*.

Klippen und explodierende Gradienten

Sehr steile Regionen (typisch bei rekurrenten Netzen und tiefen Produkten von Matrizen) katapultieren die Parameter weit weg. Gegenmittel: *Gradient Clipping*, $\mathbf{g}' = \min(1, c/\|\mathbf{g}\|) \mathbf{g}$ – Richtung beibehalten, Norm auf c begrenzen.

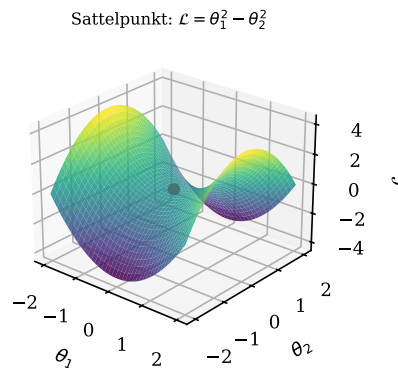


Abbildung 12: Sattelpunkt: Minimum entlang θ_1 , Maximum entlang θ_2 . Der Gradient verschwindet, obwohl der Punkt in θ_2 -Richtung instabil ist.

5.4 Momentum und Nesterov-Momentum

Momentum akkumuliert einen exponentiell gewichteten Mittelwert vergangener Gradienten und bewegt sich in dessen Richtung – wie eine Kugel mit Masse und Reibung in der Verlustlandschaft:

$$\mathbf{v} \leftarrow \alpha \mathbf{v} - \epsilon \hat{\mathbf{g}}, \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \mathbf{v}, \quad \alpha \in [0, 1). \quad (18)$$

Bei konstantem Gradienten $\mathbf{g}$ konvergiert die Geschwindigkeit gegen $\epsilon \|\mathbf{g}\| / (1 - \alpha)$; mit $\alpha = 0,9$ also das Zehnfache eines SGD-Schritts. Gradientenkomponenten, die von Schritt zu Schritt das Vorzeichen wechseln

(die steile Richtung im schlecht konditionierten Tal), heben sich im Mittel auf; konsistente Komponenten (die flache Richtung) werden verstärkt. Beides dämpft die Oszillation und beschleunigt den Fortschritt (Abb. 13, Mitte). Typische Werte: $\alpha \in \{0,5; 0,9; 0,99\}$.

Nesterov-Momentum wertet den Gradienten nicht an θ , sondern an der vorausgeschauten Position $\theta + \alpha \mathbf{v}$ aus: $\mathbf{v} \leftarrow \alpha \mathbf{v} - \epsilon \nabla \mathcal{L}(\theta + \alpha \mathbf{v})$. Das korrigiert den Schritt, bevor die Kugel über das Ziel hinausschießt; im konvexen Batch-Fall verbessert es die Konvergenzrate von $O(1/k)$ auf $O(1/k^2)$.

5.5 Adaptive Lernraten: AdaGrad, RMSProp, Adam

Die Idee: jede Parameterkomponente bekommt ihre eigene Lernrate, skaliert mit der Historie ihrer Gradientenbeträge. Parameter mit großen, häufigen Gradienten werden gebremst, seltene und kleine beschleunigt.

Adaptive Gradient Algorithm (AdaGrad) [6]

Passt die Lernrate für jeden Parameter individuell an, basierend auf der Historie der vergangenen Gradienten. Die Updates weisen folgende Struktur auf:

$$\mathbf{r} \leftarrow \mathbf{r} + \hat{\mathbf{g}} \odot \hat{\mathbf{g}} \quad (19)$$

$$\theta \leftarrow \theta - \frac{\epsilon \hat{\mathbf{g}}}{\sqrt{\mathbf{r} + \delta}} \quad (20)$$

Gut bei spärlichen Gradienten, allerdings wächst der Akkumulator $\mathbf{r}$, welcher die Summe aller bisherigen quadrierten Gradienten speichert, monoton, was die effektive Lernrate bei nicht-konvexen Problemen zu früh zu klein werden lässt.

Root Mean Square Propagation (RMSProp) [7]

Ist eine Erweiterung des AdaGrad-Optimierers, die entwickelt wurde, um das vorzeitige Einfrieren der Lernrate bei tiefen neuronalen Netzen zu verhindern. Statt der gesamten Historie speichert RMSProp nur einen exponentiell gewichteten Mittelwert der vergangenen quadrierten Gradienten. Dadurch wird die ferne Vergangenheit schrittweise "vergessen":

$$\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \hat{\mathbf{g}} \odot \hat{\mathbf{g}} \quad (21)$$

Typische Werte der Abklingrate $\rho \approx 0,9$. Die Aktualisierung der Gewichte θ erfolgt anschließend skaliert durch folgende Historie:

$$\theta \leftarrow \theta - \frac{\epsilon}{\sqrt{\mathbf{r} + \delta}} \odot \hat{\mathbf{g}} \quad (22)$$

Durch das "Vergessen" der Historie auch in nicht-stationären Landschaften funktional. Genau dieses Verhalten ist für tiefere Netze essenziell, da sich die optimale Schrittweite für die Gewichte einer Schicht ständig ändern, während sich die anderen Schichten im Netzwerk ebenfalls verändern.

Adaptive Moment Estimation (Adam) [2]

Kombiniert Momentum (erstes Moment $\mathbf{m}$) mit RMSProp (zweites Moment $\mathbf{v}$) und korrigiert beide um ihren Initialisierungsbias (algorithm 3).

Warum die Bias-Korrektur? Da $\mathbf{v}_0 = \mathbf{0}$, ist $\mathbf{v}_t = (1 - \beta_2) \sum_{i=1}^t \beta_2^{t-i} \mathbf{g}_i^2$ und bei stationärem Gradienten $\mathbb{E}[\mathbf{v}_t] = \mathbb{E}[\mathbf{g}_i^2] (1 - \beta_2^t)$. Der Faktor $(1 - \beta_2^t)$ ist für kleine t und β_2 nahe 1 winzig (bei $\beta_2 = 0,999$, $t = 10$: $\approx 0,01$). Ohne Division durch diesen Faktor wäre $\sqrt{\mathbf{v}_t}$ zu klein und die ersten Schritte um Größenordnungen zu groß – genau das, was RMSProp mit Momentum instabil macht [2, Abschn. 6.4].

Eigenschaften: Die effektive Schrittweite ist $\Delta_t = \alpha \hat{\mathbf{m}}_t / \sqrt{\hat{\mathbf{v}}_t}$. Da $|\mathbb{E}[\mathbf{g}] / \sqrt{\mathbb{E}[\mathbf{g}^2]}| \leq 1$, gilt in der Regel $|\Delta_t| \lesssim \alpha$: α ist eine obere Schranke für die Schrittgröße im Parameterraum (*Trust Region*), unabhängig von der Skala der Gradienten. Nahe einem Optimum sinkt das „Signal-Rausch-Verhältnis“ $\hat{\mathbf{m}} / \sqrt{\hat{\mathbf{v}}}$. Dadurch werden die Schritte automatisch kleiner, was einer eingebauten Form von Lernratenabfall entspricht.

Voraussetzung: Schrittweite α ; Abklingraten $\beta_1, \beta_2 \in [0, 1)$; stochastischer Loss $f(\theta)$; Start θ_0
 $m_0 \leftarrow \mathbf{0}, v_0 \leftarrow \mathbf{0}, t \leftarrow 0$
while θ_t nicht konvergiert **do**
 $t \leftarrow t + 1; \quad g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$
 $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ // erstes Moment (Mittelwert)
 $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ // zweites Rohmoment
 $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t); \quad \hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ // Bias-Korrektur
 $\theta_t \leftarrow \theta_{t-1} - \alpha \hat{m}_t / \sqrt{\hat{v}_t + \varepsilon}$
end

Algorithm 3: Adam [2, Alg. 1]. Standardwerte: $\alpha = 0,001, \beta_1 = 0,9, \beta_2 = 0,999, \varepsilon = 10^{-8}$. Alle Operationen elementweise.

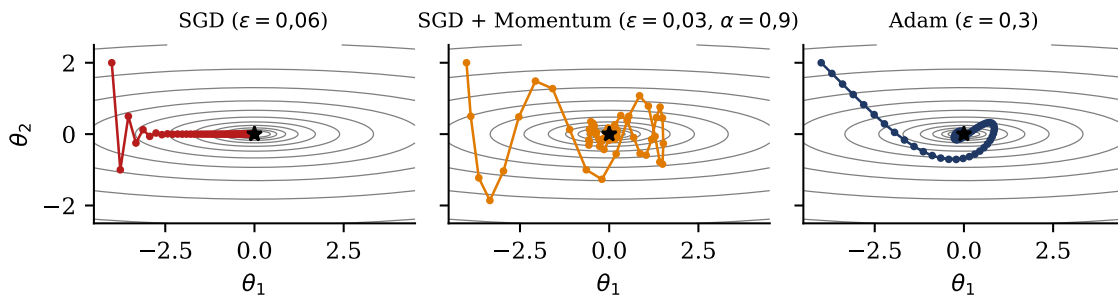


Abbildung 13: Trajektorien dreier Optimierer auf der schlecht konditionierten Quadrik $\mathcal{L} = \frac{1}{2}(\theta_1^2 + 25\theta_2^2)$ (Konditionszahl 25), Start bei $(-4, 2)$, 60 Schritte. SGD oszilliert in der steilen θ_2 -Richtung und kriecht in θ_1 . Momentum kommt schneller voran, schießt aber zunächst über. Adam normiert jede Richtung einzeln und bewegt sich mit nahezu konstanter Schrittlänge $\approx \alpha$ auf das Minimum zu.

Verfahren	Zustand pro Parameter	Hyperparameter (Default)	Typischer Einsatz
SGD	–	ϵ	Baseline; mit Plan sehr gut bei CNNs
SGD + Momentum	v	$\epsilon, \alpha = 0,9$	Standard für Bildklassifikation (ResNets)
Nesterov	v	$\epsilon, \alpha = 0,9$	wie Momentum, etwas robuster
AdaGrad	r (Summe)	$\epsilon, \delta = 10^{-7}$	konvexe/spärliche Probleme
RMSProp	r (EWMA)	$\epsilon, \rho = 0,9$	RNNs, nicht-stationär
Adam	m, v	$\alpha = 10^{-3}, \beta_1 = 0,9, \beta_2 = 0,999$	Default-Wahl; Transformer, LLM-Fine-Tuning
AdamW	m, v	wie Adam + λ	Adam mit <i>entkoppeltem</i> Weight Decay (Abschn. 6.2)

Tabelle 3: Übersicht der Optimierer. Der Zustand verdoppelt bzw. verdreifacht den Speicherbedarf für Parameter – relevant bei großen Modellen.

In der Praxis

Es gibt keinen in allen Fällen besten Optimierer [1, Abschn. 8.5.4]. Bewährte Startkonfiguration: **Adam** mit $\alpha = 10^{-3}$, dann Lernrate über $\{10^{-4}, 3 \cdot 10^{-4}, 10^{-3}, 3 \cdot 10^{-3}\}$ prüfen. SGD+Momentum erreicht bei sorgfältig eingestelltem Lernratenplan oft leicht bessere Generalisierung, braucht aber mehr Tuning.

5.6 Parameterinitialisierung

Da das Optimierungsproblem nicht konvex ist, entscheidet der Startpunkt mit über Erreichbarkeit und Qualität der Lösung sowie darüber, ob Signale überhaupt durch das Netz fließen. Zwei Prinzipien sind dabei entscheidend: (1) *Symmetriebrechung*: identisch initialisierte Einheiten erhalten identische Gradienten und bleiben für immer identisch. Daher werden zufällige Startgewichte gewählt, damit sich die Einheiten unterschiedlich entwickeln können. (2) *Varianzerhaltung* über die Schichten: Die Streuung / Varianz der Werte sollte beim Durchlaufen der Schichten weder immer kleiner noch immer größer werden. Andernfalls verschwinden oder explodieren die Signale (und mit ihnen die Gradienten) und das Netz kann nicht sinnvoll lernen.

Betrachte eine lineare Einheit die einfach eine gewichtete Summe ihrer Eingänge berechnet $o_i = \sum_{j=1}^{n_{in}} w_{ij}x_j$ mit Gewichten $w_{ij} \sim \mathcal{N}(0, \sigma^2)$, $\mathbb{E}[x_j] = 0$, $\mathbb{V}[x_j] = \gamma^2$, unabhängig. Dann ist $\mathbb{E}[o_i] = 0$ und

$$\mathbb{V}[o_i] = \sum_{j=1}^{n_{in}} \mathbb{E}[w_{ij}^2] \mathbb{E}[x_j^2] = n_{in} \sigma^2 \gamma^2. \quad (23)$$

Damit die Aktivierungen weder explodieren noch verschwinden, muss $n_{in}\sigma^2 = 1$ gelten; analog für den Rückwärtsdurchlauf $n_{out}\sigma^2 = 1$. In der Regel können nicht beide Bedingungen gleichzeitig exakt erfüllt werden, da $n_{in} \neq n_{out}$ gelten kann. Deshalb wird als Kompromiss der Mittelwert der geforderten Varianzen

$$\sigma^2 = 2/(n_{in} + n_{out}) \quad (24)$$

, die sog. **Glorot-**, bzw. **Xavier-Initialisierung** [8] genutzt. Für den Sonderfall ReLU, welche im Mittel die Hälfte der Aktivierungen auf 0 setzt, verdoppelt man die Varianz:

$$\sigma^2 = 2/n_{in} \quad (25)$$

, die **He-Initialisierung** [9]. Falls eine Gleichverteilung mit $\text{Unif}(-a, a)$ anstelle einer Normalverteilung genutzt wird gilt der bekannte Zusammenhang $\sigma^2 = a^2/3$. Löst man nach a auf und setzt die Glorot-Varianz ein ergibt sich:

$$a = \sqrt{\frac{6}{n_{in} + n_{out}}} \quad (26)$$

Biases werden meist auf 0 initialisiert, da sie (anders als die Gewichte) keine Symmetrieberechnung benötigen.

In der Praxis

Empfehlung: Glorot für tanh/Sigmoid/Softmax-Schichten, He für ReLU-Familie, LeCun ($\sigma^2 = 1/n_{in}$) für SELU. `nn.Linear` in PyTorch initialisiert standardmäßig mit einer He-artigen Gleichverteilung; wer explizit kontrollieren will, nutzt `nn.init.kaiming_normal_` bzw. `xavier_uniform_`. Wie stark die Wahl bei tiefen Netzen wirkt, zeigt Abb. 17.

6 Regularisierung

Ein MLP mit 10^5 Parametern kann 60 000 MNIST-Bilder auswendig lernen. Regularisierung umfasst alle Maßnahmen, die den *Test*-Fehler senken, auch wenn sie den *Trainings*-Fehler erhöhen [1, Kap. 7]. Die wichtigsten Familien: Strafterme auf Parametern, frühes Stoppen, Rauschen (Dropout, Dataaugmentation) und Normalisierung. Der Abschnitt schließt mit einem Entscheidungsleitfaden.

6.1 Overfitting erkennen

Um Overfitting zu erkennen wird ein sog. *Validierungssplit* eingesetzt: Ein Teil der Trainingsdaten (typisch 10–20 %) wird nie zum Training verwendet, sondern nur zur Bewertung. Öffnet sich zwischen Trainings- und Validierungsverlust eine Schere, passt das Modell Rauschen an (Abb. 14). Ein weiterer Teil der Trainingsdaten, der *Test-Split* bleibt für die finale, einmalige Bewertung reserviert. Wird der Testsplitt stattdessen schon zur Auswahl der Hyperparameter genutzt, fließt dessen Information indirekt ins Modell ein und die damit gemessene Generalisierung fällt dann zu optimistisch aus.

6.2 Parameternormen: Weight Decay und L1

Klassischer Weise wird der Verlustfunktion ein Strafterm hinzugefügt, welcher große Gewichte bestraft:

$$\tilde{\mathcal{L}}(\theta) = \mathcal{L}(\theta) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2 \implies \mathbf{w} \leftarrow \mathbf{w} - \epsilon(\nabla_{\mathbf{w}} \mathcal{L} + \lambda \mathbf{w}) = (1 - \epsilon\lambda)\mathbf{w} - \epsilon \nabla_{\mathbf{w}} \mathcal{L}. \quad (27)$$

Jeder Schritt schrumpft die Gewichte multiplikativ um $(1 - \epsilon\lambda)$ (daher „Weight Decay“). Bayesianisch entspricht dies der Annahme, die Gewichte kämen aus einer einem Gaußschen-Verteilung $\mathcal{N}(\mathbf{w} | \mathbf{0}, \lambda^{-1} \mathbf{I})$ und man sucht die wahrscheinlichste Erklärung dafür (MAP-Schätzung). Der Effekt in Richtung der Hesse-Eigenvektoren: Komponenten mit großer Krümmung (der Loss „bestimmt“ sie) bleiben nahezu erhalten, Komponenten mit kleiner Krümmung (der Loss „ist gleichgültig“) werden gegen null gezogen. Biases werden üblicherweise nicht bestraft.

Die alternative L^1 -Strafe $\lambda \|\mathbf{w}\|_1$ drückt Gewichte nicht nur klein, sondern exakt auf 0 und erzeugt so *spärliche* Lösungen. Bei tiefen Netzen ist sie weniger verbreitet, da GPUs von dieser Art unstrukturierter Spärlichkeit nicht profitieren.

In der Praxis

Bei adaptiven Optimierern ist die Kopplung in Gl. (27) problematisch: Adam skaliert den Term $\lambda \mathbf{w}$ mit $1/\sqrt{\hat{v}}$, sodass Parameter mit großen Gradienten kaum regularisiert werden. **AdamW** [10] wendet das Schrumpfen $(1 - \epsilon\lambda)\mathbf{w}$ *entkoppelt* vom adaptiven Schritt an. In PyTorch: `torch.optim.AdamW(..., weight_decay=0.01)`; bei Adam bedeutet `weight_decay` die gekoppelte L^2 -Variante.

6.3 Early Stopping

Die einfachste und günstigste Regularisierung. Der Verlauf des Validierungsfehlers wird beobachtet und die Parameter des besten Zeitpunkts werden behalten (Abb. 14). Man wartet dabei eine *Geduldsspanne* (patience) von einigen Epochen ohne Verbesserung ab, bevor abgebrochen wird. Early Stopping wirkt, weil die Zahl der Gradientenschritte τ mit Lernrate ϵ den erreichbaren Abstand vom Startpunkt begrenzt; für ein quadratisches Modell ist es äquivalent zu L^2 -Regularisierung mit $\lambda \approx 1/(\tau\epsilon)$ [1, Abschn. 7.8]. Der Hyperparameter „Trainingsdauer“ wird dabei kostenlos mitbestimmt.

6.4 Dropout

Definition 6.1: Dropout []

Während des Trainings wird für jedes Beispiel und jeden Vorwärtsthroughlauf jede verdeckte Einheit mit Wahrscheinlichkeit p auf null gesetzt (alle ausgehenden Verbindungen entfernt). Formal: $\tilde{\mathbf{z}}_l = \epsilon_l \odot \mathbf{z}_l$ mit $\epsilon_{li} \sim \text{Ber}(1 - p)$. Zur Testzeit ist das Rauschen aus; damit die erwartete Eingangsaktivierung gleich bleibt, werden die Gewichte mit $(1 - p)$ skaliert (*weight scaling*) – oder, äquivalent und in PyTorch üblich, bereits im Training mit $1/(1 - p)$ hochskaliert (*inverted dropout*).

Zwei Lesarten erklären, warum das funktioniert: (1) *Ko-Adaptation verhindern*: keine Einheit kann sich auf das Vorhandensein bestimmter anderer verlassen, jede muss für sich nützlich sein. (2) *Ensemble*: Dropout trainiert implizit exponentiell viele Teilnetze mit geteilten Gewichten, sodass die Gewichtsskalierung zur Testzeit das geometrische Mittel ihrer Vorhersagen approximiert. Lässt man Dropout auch zur

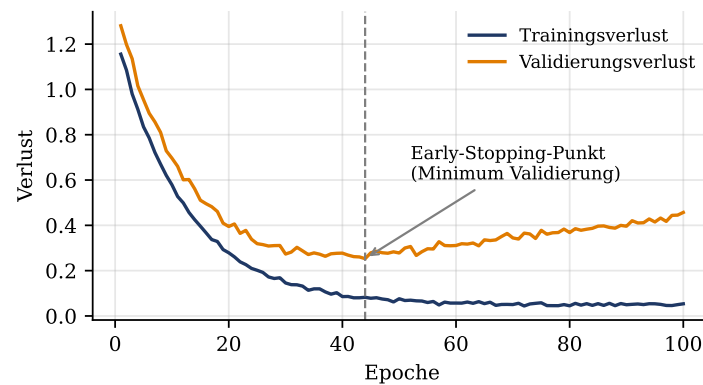


Abbildung 14: Early Stopping (synthetischer Verlauf). Der Trainingsverlust sinkt monoton, der Validierungsverlust erreicht nach etwa 45 Epochen sein Minimum und steigt danach: Das Modell beginnt, Rauschen zu lernen. Behalten wird der Parameterstand am Minimum.

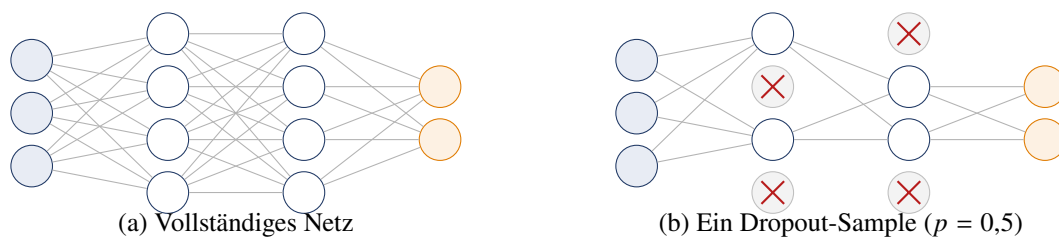


Abbildung 15: Dropout entfernt in jedem Vorwärtsthroughlauf zufällig Einheiten. Jedes Sample ist ein anderes, ausgedünntes Teilnetz mit geteilten Gewichten (nach [11, Abb. 1]).

Testzeit an und mittelt über S Vorwärtsthroughläufe (*Monte-Carlo-Dropout*), erhält man eine Schätzung der Vorhersageunsicherheit..

In der Praxis

Typisch $p = 0,5$ für verdeckte Schichten, $p \approx 0,2$ für die Eingabe. Dropout wird *im Anschluss* an die Aktivierung implementiert (`nn.Linear` $\rightarrow$ `nn.ReLU` $\rightarrow$ `nn.Dropout(p)`). Vergessen Sie `model.eval()` vor der Evaluation nicht, da sonst der Trainingsmodus des Modells aktiv bleibt und die Vorhersage rauscht. Bei Batch Normalization und in modernen CNNs ist Dropout weitgehend durch andere Mechanismen ersetzt worden. In MLPs und in den Feedforward-Blöcken von Transformern bleibt Dropout der Standard.

6.5 Rauschen und Dataaugmentation

Der beste Regularisierer sind mehr Daten. Wenn man keine hat, können synthetische Daten selbst erzeugt werden. Beispielsweise können neue Daten bei Bildern durch Verschiebung, Rotation, Skalierung, Helligkeitsänderung, bei Audio durch Zeitdehnung, bei Text durch Synonymersetzung erzeugt werden. Voraussetzung ist, dass die Transformation das Label nicht ändert (eine Spiegelung macht aus einer 6 keine 9, aber aus einem „b“ ein „d“). Additives Rauschen auf Eingaben oder Gewichten wirkt ähnlich. *Label Smoothing* (Ziel $1 - \epsilon$ statt 1 für die wahre Klasse) verhindert, dass Softmax-Logits gegen unendlich getrieben werden.

6.6 Batch Normalization

Batch Normalization [12] ist streng genommen keine Regularisierung, sondern eine Reparametrisierung, welche mittels regularisierendem Nebeneffekt die Optimierung erleichtert. Das Problem: Ein Gradientenschritt ändert alle Schichten gleichzeitig unter der Annahme, die restlichen Schichten blieben unverändert. In tiefen Netzen verschiebt sich dadurch die Verteilung der Eingaben jeder Schicht ständig. Damit diese

verschiebung nicht außer Kontrolle gerät müssen sehr kleine Lernraten genutzt werden.

Definition 6.2: Batch Normalization

Für einen Minibatch $\mathbf{H} \in \mathbb{R}^{m \times K}$ von Prä-Aktivierungen einer Schicht (Zeilen = Beispiele) wird jede Spalte k standardisiert und anschließend mit lernbaren Parametern γ_k, β_k neu skaliert:

$$\mu_k = \frac{1}{m} \sum_i H_{ik}, \quad \sigma_k^2 = \frac{1}{m} \sum_i (H_{ik} - \mu_k)^2, \quad \hat{H}_{ik} = \frac{H_{ik} - \mu_k}{\sqrt{\sigma_k^2 + \delta}}, \quad \text{BN}(H_{ik}) = \gamma_k \hat{H}_{ik} + \beta_k. \quad (28)$$

Entscheidend: μ_k und σ_k sind Teil des Berechnungsgraphen – Backprop läuft *durch* die Normalisierung. Der Optimierer kann daher Mittelwert und Varianz einer Schicht nicht mehr über die Gewichte verschieben, sondern nur noch über γ, β : eine Parametrisierung, die für Gradientenabstieg deutlich günstiger konditioniert ist.

Zur Testzeit gibt es keinen Batch; stattdessen werden während des Trainings laufende Mittelwerte von μ_k und σ_k^2 gepflegt und eingesetzt (`model.eval()` schaltet um). Da die Statistik pro Minibatch rauscht, wirkt BN zusätzlich wie ein leichtes Rauschen (der Eingangs erwähnte regularisierende Nebeneffekt). BN erlaubt deutlich höhere Lernraten, macht das Training robuster gegenüber der Initialisierung und ist in CNNs Standard („Conv → BN → ReLU“). Für Transformer wird Layer Normalization verwendet, die über die Merkmale eines einzelnen Beispiels statt über den Batch normiert und daher batchgrößenunabhängig ist.

6.7 Implizite Regularisierung durch SGD

Auch ohne explizite Maßnahme regularisiert Minibatch-SGD. Das Gradientenrauschen verhindert, dass der Optimierer in schmale, scharfe Minima „hineinfällt“, und bevorzugt breite, flache Minima (Abb. 16), die robuster gegen Parameterstörungen sind und besser generalisieren [13], [14]. Eine Analyse im Grenzfalle kontinuierlicher Zeit [15] zeigt, dass SGD effektiv den modifizierten Verlust $\mathcal{L}(\mathbf{w}) + \frac{\epsilon}{4} \sum_k \|\nabla \mathcal{L}_k(\mathbf{w})\|^2$ minimiert, in dem die Varianz der Minibatch-Gradienten bestraft wird. Die praktische Folge ist, dass sehr große Batches mit fester Lernrate diesen Effekt verlieren. Daher gilt die Faustregel, dass die Lernrate proportional zur Batchgröße erhöht wird.

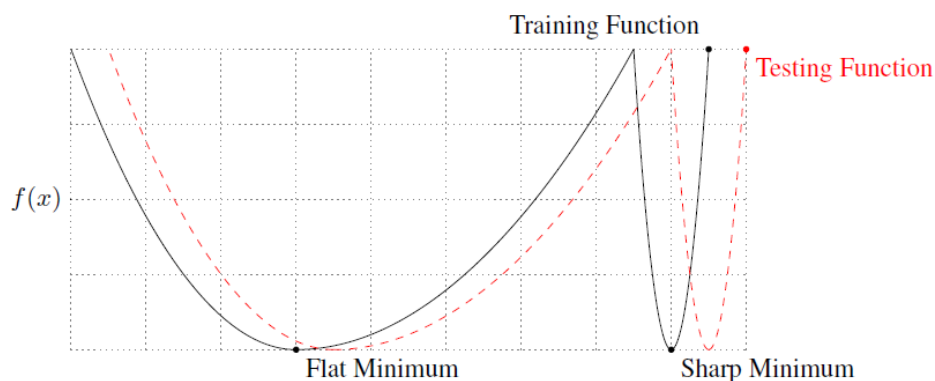


Abbildung 16: Flache vs. scharfe Minima [14, Abb. 1]. Verschiebt sich die Verlustfläche beim Übergang von Training zu Test leicht, bleibt der Verlust im flachen Minimum fast unverändert. Beim scharten Minimum dagegen führt selbst diese kleine Verschiebung zu einem starken Anstieg des Verlustes. SGD-Rauschen findet bevorzugt flache Minima.

6.8 Entscheidungsleitfaden: Welche Regularisierung wann?

7 Vanishing und Exploding Gradients

Maßnahme	Wann sinnvoll	Kosten / Risiken	Hyperparameter
Early Stopping	immer ; erste Maßnahme	Val.-split nötig; ansonsten gratis	patience
Weight Decay	Standard bei fast allen Modellen	zu groß $\Rightarrow$ Underfitting	$\lambda \in [10^{-5}, 10^{-2}]$
Dropout	MLPs, Transformer-FFN; viele Parameter, wenig Daten	längeres Training; nicht mit BN kombinieren; eval() nicht vergessen	$p \in [0,1; 0,5]$
Datenaugmentation	Bilder, Audio; wann immer labelerhaltende Transformationen existieren	Domänenwissen nötig; falsche Augmentation schadet	Art/Stärke
Batch Norm	tiefe CNNs/MLPs; instabiles Training	Batchgröße $\gtrsim 16$; Train/Test-Unterschied	Momentum der laufenden Statistik
Label Smoothing	Klassifikation mit überkonfidenten Modellen	kalibrierte Wahrscheinlichkeiten leicht verzerrt	$\varepsilon \approx 0,1$
Kleineres Modell	wenn alles andere versagt	Kapazität begrenzt	Breite/Tiefe

Tabelle 4: Entscheidungsleitfaden Regularisierung. Reihenfolge in der Praxis: Early Stopping $\rightarrow$ Weight Decay $\rightarrow$ Augmentation (falls möglich) $\rightarrow$ Dropout/BN $\rightarrow$ Architektur.

7.1 Analyse

Der Adjungierte an Schicht l entsteht aus dem an Schicht $l+1$ durch Multiplikation mit der Jacobi-Matrix $J_l = \partial \mathbf{z}_{l+1} / \partial \mathbf{z}_l$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_l} = J_l^\top \frac{\partial \mathcal{L}}{\partial \mathbf{z}_{l+1}} \implies \frac{\partial \mathcal{L}}{\partial \mathbf{z}_l} = J_l^\top J_{l+1}^\top \cdots J_{L-1}^\top \frac{\partial \mathcal{L}}{\partial \mathbf{z}_L}. \quad (29)$$

Wären alle $J_l = J$ gleich, stünde hier $(J^\top)^{L-l}$. Das Verhalten einer Matrixpotenz wird vom Spektralradius $\rho(J)$ (größter Eigenwertbetrag) bestimmt: Für $\rho > 1$ wächst der Gradient exponentiell mit der Tiefe (*exploding*), für $\rho < 1$ schrumpft er exponentiell (*vanishing*). Für ein MLP ist $J_l = \text{diag}(\varphi'(\mathbf{a}_{l+1})) \mathbf{W}_{l+1}$, das Produkt aus der Ableitung der Aktivierungsfunktion und der Gewichtsmatrix. Beide Faktoren können das Signal dämpfen: Sigmoid-Ableitungen sind $\leq 0,25$, und schlecht skalierte Gewichte multiplizieren mit $\|\mathbf{W}\| \neq 1$.

Abbildung 17 zeigt das Ergebnis eines numerischen Experiments: 30 vollverbundene Schichten à 100 Einheiten, zufällige Eingabe, Gradientennorm pro Schicht.

Merke

Vanishing Gradients sind kein Bug im Optimierer, sondern eine Eigenschaft tiefer *multiplikativer* Kompositionen. Jede Gegenmaßnahme greift an einem der Faktoren in Gl. (29) an: an φ' (Aktivierung), an $\mathbf{W}$ (Initialisierung, Normalisierung) oder an der multiplikativen Struktur selbst (Residualverbindungen).

7.2 Gegenmaßnahmen

1. **Nicht sättigende Aktivierungen** (Tab. 2): ReLU hat $\varphi' = 1$ für aktive Einheiten. Das Restrisiko „tote“ Einheiten adressieren Leaky ReLU, ELU, GELU.
2. **Varianzerhaltende Initialisierung** (Abschn. 5.6): Glorot/He sorgen dafür, dass $\|J_l\| \approx 1$ zu Beginn des Trainings.
3. **Normalisierung** (Abschn. 6.6): Batch/Layer Norm hält die Aktivierungsstatistik jeder Schicht *während* des Trainings stabil, nicht nur zu Beginn.
4. **Residualverbindungen** [16]: Statt $\mathbf{z}_{l+1} = F_l(\mathbf{z}_l)$ wird $\mathbf{z}_{l+1} = \mathbf{z}_l + F_l(\mathbf{z}_l)$ gelernt. Dann gilt für beliebige $l < L$

$$\mathbf{z}_L = \mathbf{z}_l + \sum_{i=l}^{L-1} F_i(\mathbf{z}_i) \implies \frac{\partial \mathcal{L}}{\partial \mathbf{z}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_L} \left(\mathbf{I} + \sum_{i=l}^{L-1} \frac{\partial F_i}{\partial \mathbf{z}_l} \right). \quad (30)$$

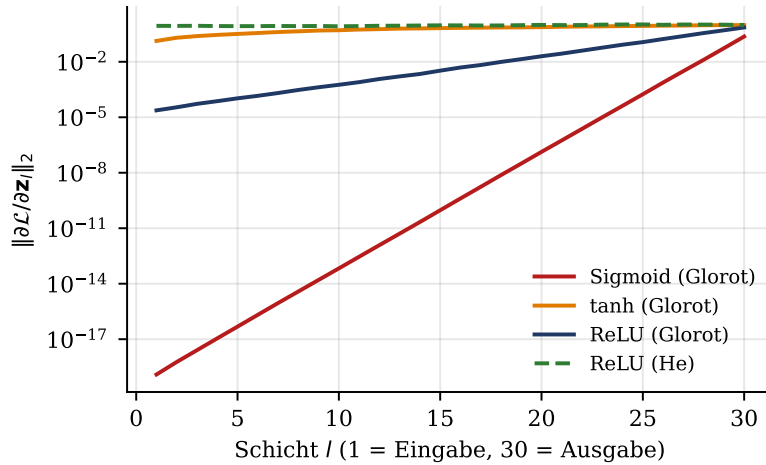
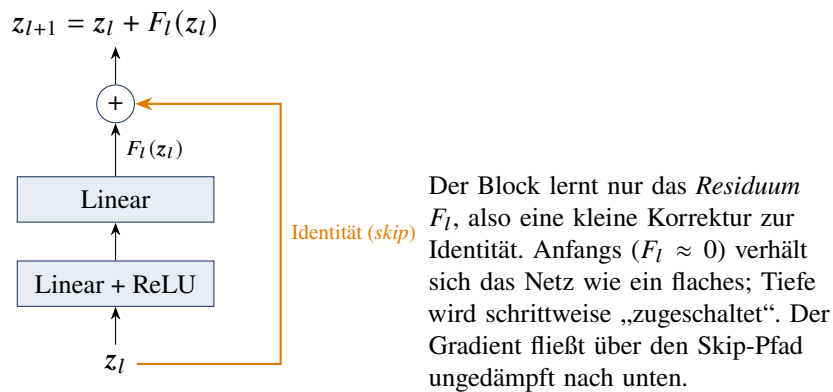


Abbildung 17: Gradientennorm $\|\partial \mathcal{L} / \partial z_l\|_2$ über die Tiefe eines 30-schichtigen MLPs (100 Einheiten je Schicht, Mittel über 20 zufällige Initialisierungen, log-Skala). Mit Sigmoid schrumpft der Gradient um 17 Größenordnungen und die ersten Schichten lernen praktisch nichts. ReLU mit Glorot-Initialisierung verliert noch einen Faktor $\approx 2^{-15}$, weil im Mittel die Hälfte der Einheiten inaktiv ist; He-Initialisierung ($\sigma^2 = 2/n$) korrigiert genau das. tanh bleibt dank Steigung 1 um 0 und Nullzentrierung stabil, solange die Prä-Aktivierungen klein sind.

Der Term I liefert einen direkten, ungedämpften Pfad vom Ausgang zu jeder Schicht: Die Komposition wird *additiv* statt *multiplikativ*. Das ist der Grund, warum Netze mit hunderten Schichten trainierbar sind (Abb. 18). Das Prinzip ist identisch in Transformern.

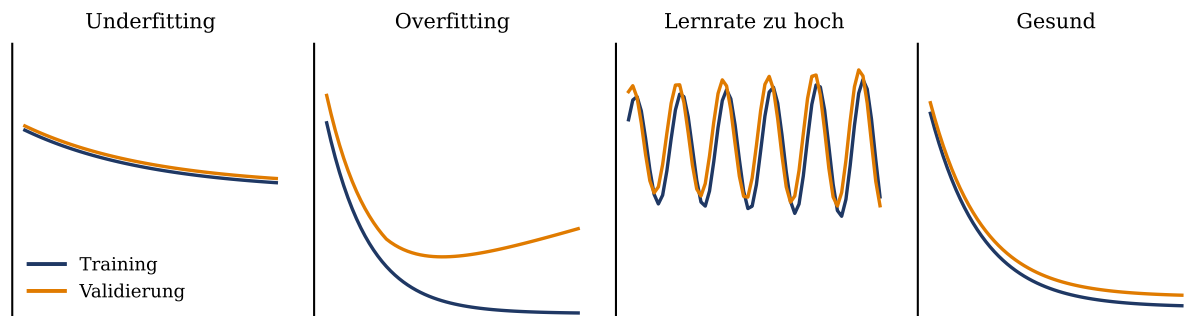
5. **Gradient Clipping** gegen die explodierende Seite (Abschn. 5.3); in PyTorch zwischen `backward()` und `step()`:
- ```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm).
```



**Abbildung 18:** Residualblock. Die Identitätsverbindung macht aus der multiplikativen eine additive Gradientenstruktur (Gl. (30)).

## 8 Trainingsverläufe interpretieren

Alles Vorangegangene läuft auf eine Fähigkeit hinaus: aus zwei Kurven, dem Trainings- und Validierungsverlust (bzw. -genauigkeit) über die Epochen, die richtige Diagnose zu stellen und die passende Stellschraube zu wählen. Abbildung 19 zeigt die vier Grundmuster, Tab. 5 ordnet Symptome, Ursachen und Maßnahmen zu.



**Abbildung 19:** Vier typische Trainingsverläufe (schematisch). *Underfitting:* beide Kurven hoch und nah beieinander. *Overfitting:* Schere zwischen Training und Validierung. *Lernrate zu hoch:* Oszillation, kein Fortschritt. *Gesund:* beide sinken, kleiner konstanter Abstand.

| Symptom                                                                            | Wahrscheinliche Ursache                                                                                                                                 | Maßnahmen (in Reihenfolge)                                                                                                      |
|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Trainingsverlust sinkt nicht / bleibt bei $\log C$ ( $\approx 2,30$ bei $C = 10$ ) | Lernrate zu klein; Gradienten verschwinden; Bug (Loss auf Wahrscheinlichkeiten statt Logits, falsche Label-Codierung, <code>zero_grad</code> vergessen) | Bug ausschließen (ein Batch überfitten – muss auf $\approx 0$ gehen); Lernrate $\times 10$ ; Aktivierung/Initialisierung prüfen |
| Verlust wird NaN / explodiert                                                      | Lernrate zu groß; explodierende Gradienten; numerisch instabiler Loss                                                                                   | Lernrate $/10$ ; Gradient Clipping; <code>CrossEntropyLoss</code> statt manueller Softmax+Log                                   |
| Verlust oszilliert stark ohne Trend                                                | Lernrate zu groß; Batch zu klein                                                                                                                        | Lernrate senken; Batch vergrößern; Lernratenabfall                                                                              |
| Training gut, Validierung deutlich schlechter (Schere)                             | Overfitting; zu große Kapazität für die Datenmenge                                                                                                      | Early Stopping; Weight Decay; Augmentation; Dropout; kleineres Modell; mehr Daten                                               |
| Beide Kurven hoch und flach                                                        | Underfitting; Modell zu klein; zu starke Regularisierung; zu wenig Epochen                                                                              | Regularisierung reduzieren; breiter/tiefer; länger trainieren; Lernrate erhöhen                                                 |
| Validierung besser als Training                                                    | Dropout/BN aktiv im Training, inaktiv in <code>eval()</code> ; Trainingsverlust als laufendes Mittel über die Epoche berechnet                          | normal, kein Handlungsbedarf – ggf. Trainingsverlust nach der Epoche neu evaluieren                                             |
| Plötzlicher Sprung nach unten                                                      | Lernratenabfall (step decay) hat gegriffen                                                                                                              | erwartetes Verhalten                                                                                                            |
| Genauigkeit gut, Verlust steigt                                                    | Modell wird überkonfident bei wenigen Fehlern                                                                                                           | Label Smoothing; Weight Decay; früher stoppen                                                                                   |

**Tabelle 5:** Diagnose-Tabelle für Trainingsverläufe.

### In der Praxis

#### Debug-Protokoll vor jedem Experiment.

1. Verlust beim ersten Batch prüfen: Bei  $C$  Klassen und zufälliger Initialisierung erwartet man  $\mathcal{L} \approx \ln C$ .
  2. Einen einzelnen Minibatch überfitten: Der Verlust muss gegen 0 gehen. Tut er das nicht, ist es kein Hyperparameter-, sondern ein Codeproblem.
  3. Lernrate über Zehnerpotenzen rastern, alles andere fest.
  4. Erst dann Architektur und Regularisierung variieren – und immer nur eine Variable zugleich.
- Loggen Sie jeden Lauf (Konfiguration, Kurven, Seed).

## Literatur

- [1] I. Goodfellow, Y. Bengio und A. Courville, *Deep learning* (Adaptive computation and machine learning), en. Cambridge, Massachusetts: The MIT Press, 2016, ISBN: 978-0-262-03561-3.
- [2] D. P. Kingma und J. Ba, *Adam: A Method for Stochastic Optimization*, en, arXiv:1412.6980 [cs.LG], 2015. DOI: [10.48550/arXiv.1412.6980](https://doi.org/10.48550/arXiv.1412.6980). Adresse: <http://arxiv.org/abs/1412.6980>.
- [3] C. M. Bishop und H. Bishop, *Deep Learning: Foundations and Concepts*, en. Cham: Springer International Publishing, 2024, ISBN: 978-3-031-45467-7 978-3-031-45468-4. DOI: [10.1007/978-3-031-45468-4](https://doi.org/10.1007/978-3-031-45468-4). Adresse: <https://link.springer.com/10.1007/978-3-031-45468-4>.
- [4] H. Lee, R. Grosse, R. Ranganath und A. Y. Ng, „Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations,“ en, in *Proceedings of the 26th Annual International Conference on Machine Learning*, Montreal Quebec Canada: ACM, Juni 2009, S. 609–616, ISBN: 978-1-60558-516-1. DOI: [10.1145/1553374.1553453](https://doi.org/10.1145/1553374.1553453). Adresse: <https://dl.acm.org/doi/10.1145/1553374.1553453>.
- [5] I. J. Goodfellow u. a., *Generative Adversarial Networks*, en, arXiv:1406.2661 [stat.ML], Juni 2014. DOI: [10.48550/arXiv.1406.2661](https://doi.org/10.48550/arXiv.1406.2661). Adresse: <http://arxiv.org/abs/1406.2661>.
- [6] J. Duchi, E. Hazan und Y. Singer, „Adaptive Subgradient Methods for Online Learning and Stochastic Optimization,“ en,
- [7] G. Hinton und T. Tieleman, „Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude,“ en, Nr. 4, S. 26–31, 2012.
- [8] X. Glorot und Y. Bengio, „Understanding the difficulty of training deep feedforward neural networks,“ en, 2010.
- [9] K. He, X. Zhang, S. Ren und J. Sun, *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*, en, arXiv:1502.01852 [cs.CV], Feb. 2015. DOI: [10.48550/arXiv.1502.01852](https://doi.org/10.48550/arXiv.1502.01852). Adresse: <http://arxiv.org/abs/1502.01852>.
- [10] I. Loshchilov und F. Hutter, *Decoupled Weight Decay Regularization*, en, arXiv:1711.05101 [cs.LG], Jan. 2019. DOI: [10.48550/arXiv.1711.05101](https://doi.org/10.48550/arXiv.1711.05101). Adresse: <http://arxiv.org/abs/1711.05101>.
- [11] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever und R. Salakhutdinov, „Dropout: A Simple Way to Prevent Neural Networks from Overfitting,“ en, 2014.
- [12] S. Ioffe und C. Szegedy, *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*, en, arXiv:1502.03167 [cs.LG], März 2015. DOI: [10.48550/arXiv.1502.03167](https://doi.org/10.48550/arXiv.1502.03167). Adresse: <http://arxiv.org/abs/1502.03167>.
- [13] S. Hochreiter und J. Schmidhuber, „Long Short-Term Memory,“ en, *Neural Computation*, Jg. 9, Nr. 8, S. 1735–1780, Nov. 1997, ISSN: 0899-7667, 1530-888X. DOI: [10.1162/neco.1997.9.8.1735](https://doi.org/10.1162/neco.1997.9.8.1735). Adresse: <https://direct.mit.edu/neco/article/9/8/1735-1780/6109>.
- [14] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy und P. T. P. Tang, *On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima*, en, arXiv:1609.04836 [cs.LG], Feb. 2017. DOI: [10.48550/arXiv.1609.04836](https://doi.org/10.48550/arXiv.1609.04836). Adresse: <http://arxiv.org/abs/1609.04836>.
- [15] S. L. Smith, B. Dherin, D. G. T. Barrett und S. De, *On the Origin of Implicit Regularization in Stochastic Gradient Descent*, en, arXiv:2101.12176 [cs.LG], Jan. 2021. DOI: [10.48550/arXiv.2101.12176](https://doi.org/10.48550/arXiv.2101.12176). Adresse: <http://arxiv.org/abs/2101.12176>.
- [16] K. He, X. Zhang, S. Ren und J. Sun, *Identity Mappings in Deep Residual Networks*, en, arXiv:1603.05027 [cs.CV], Juli 2016. DOI: [10.48550/arXiv.1603.05027](https://doi.org/10.48550/arXiv.1603.05027). Adresse: <http://arxiv.org/abs/1603.05027>.